



MARS-SQL: A Multi-Agent Reinforcement Learning Framework For Text-To-SQL

Haolin Yang^{*1} Jipeng Zhang^{*2} Zhitao He¹ Alexander Zhou³ Yi R. (May) Fung¹

Abstract

Large Language Models (LLMs) often struggle with the precise logic and schema alignment required for complex Text-to-SQL tasks. While current methods rely heavily on static prompting, they lack the ability to dynamically adapt and self-correct through environmental interaction. To bridge this gap, we propose **MARS-SQL**, a trainable multi-agent framework for Text-to-SQL. Rather than introducing a new standalone SQL primitive, **MARS-SQL** makes an agentic workflow trainable by decomposing the problem into three specialized roles: schema grounding, query generation, and solution validation. Central to our approach is a generation agent trained via a multi-turn RL policy within a ReAct-style loop. The agent learns to iteratively reason, execute intermediate SQL actions on a live database, and refine its strategy based on execution feedback. To improve robustness, we further introduce a validation mechanism that treats solution selection as a generative modeling task, identifying the optimal interaction trajectory through next-token prediction probabilities. Empirical evaluations demonstrate the effectiveness of coupling interactive learning with trajectory ranking. **MARS-SQL** achieves state-of-the-art performance, recording an execution accuracy of 77.84% on the BIRD development dataset and 89.75% on the Spider test dataset, while also transferring strongly to out-of-domain benchmarks. Code is available at <https://github.com/YangHaolin0526/MARS-SQL>.

1. Introduction

Translating natural language questions into executable Structured Query Language (SQL) is an essential task that allows non-expert users to access structured data (Li et al., 2024a; 2023). Recent Large Language Models (LLMs) can generate simple queries for well-organised academic benchmarks. However, they often struggle with the complexity of real-world enterprise databases (Hong et al., 2025; Lei et al., 2025). To bridge this gap and tackle the challenges of interacting with complex, real-world databases, researchers have started developing SQL agents (Li et al., 2025c; Wang et al., 2025b; Li et al., 2025b). Instead of producing a query in one step, an SQL agent allows an LLM to interact with the database through multiple rounds of reasoning and feedback. This interactive process resembles how human analysts explore data, making it a more natural and effective way to handle complex database tasks.

Current methodologies in the broader field of AI agents have explored several distinct avenues. A prominent strategy is the use of multi-agent systems, where a complex task is decomposed into specialized sub-tasks, each handled by a dedicated agent (Chang et al., 2025; Wang et al., 2025c; Liu et al., 2025; Huang et al., 2025a). A parallel line of work uses test-time scaling methods that generate multiple candidate queries and then select the best one (Ni et al., 2023; Li et al., 2022). In the specific domain of Text-to-SQL, these methodologies manifest in two primary forms. One approach relies on monolithic models, which handle schema comprehension, logical planning, and SQL generation in a single pass (Pourreza et al., 2025b; Li et al., 2024b). Another prominent approach involves multi-agent frameworks that improve modularity by using API calls to closed-source LLMs, where different agent roles are defined mainly through prompting (Pourreza et al., 2025a; Gao et al., 2024; Xie et al., 2025a).

MARS-SQL addresses three main challenges facing SQL agents: (i) **Compositional reasoning**: Agents often struggle to formulate and maintain a coherent long-term plan required for complex queries. For example, they may fail to correctly combine multiple SQL clauses like joins and subqueries. This often results in a loop of fixing mi-

¹Hong Kong University of Science and Technology ²Microsoft Research ³Hong Kong Polytechnic University. Correspondence to: Yi R. (May) Fung <yrfung@ust.hk>.

nor syntax without ever addressing the flawed high-level logic. (Chaturvedi et al., 2025). To tackle this, we introduce a **Generation Agent** trained via interactive reinforcement learning, which adopts a multi-turn reasoning process to decompose complex goals and refine the query step-by-step. (ii) **Schema understanding**: Navigating schemas with massive amounts of irrelevant tables and columns (distractors) is prone to error. An agent might, for instance, hallucinate columns or misidentify join keys due to the large search space, leading to unproductive interactions (Deng et al., 2025). We address this via a specialised **Grounding Agent** that performs reasoning-driven schema identification, filtering out distractors to provide a precise schema context for generation. This role separation is especially helpful when schema grounding and SQL generation would otherwise compete for the same context budget. (iii) **Environmental grounding**: While interactivity is central to agents, current models often lack the nuanced ability to fully leverage environmental feedback (Huang et al., 2025c). They struggle to diagnose specific SQL dialect errors or recover from ambiguous execution outcomes, limiting their self-correction capabilities (Zhang et al., 2025a). **MARS-SQL** addresses this via a dual strategy: a **Generation Agent** that leverages continuous interaction to decompose complex problems and execute intermediate queries to guide its reasoning, and a **Validation Agent** that executes and verifies the final trajectories against the environment.

While recent multi-agent frameworks have explored decomposing Text-to-SQL into modular stages (Gorti et al., 2025; Gao et al., 2024), these approaches primarily rely on static prompting or supervised fine-tuning of distinct modules. **MARS-SQL** distinguishes itself by introducing a novel interactive training paradigm. Rather than relying on architectural decomposition alone, we empower the Generation Agent with a multi-turn Reinforcement Learning (RL) policy trained via Group Relative Policy Optimization (GRPO) with sparse execution rewards. This allows the agent to learn how to navigate the schema and self-correct errors dynamically, rather than merely following a prompted chain of thought. The core innovation lies in the Generation Agent’s ability to operate within a ReAct-style Think-Act-Observe loop (Yao et al., 2022b). As detailed in Figure 1, this process transforms SQL generation into a dynamic investigation. For instance, when the agent initially attempts to join the hallucinated “fprm” table, it encounters an `OperationalError`. Rather than failing silently, the agent explicitly interprets this feedback, diagnoses the schema error, and self-corrects to identify the correct “frpm” table. To further improve robustness, multiple reasoning trajectories are explored during inference, and the Validation Agent selects the optimal result. Our deployed system uses three specialized 7B agents, corresponding to an inference-time footprint of approximately 21B parameters. Through

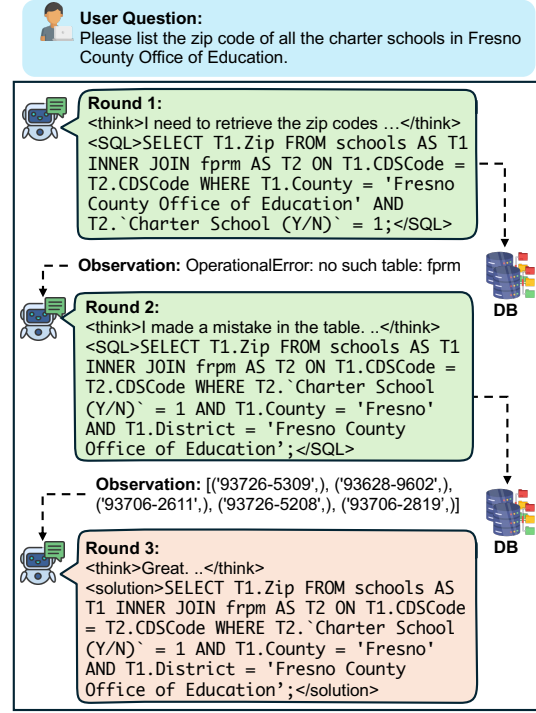


Figure 1. Illustration of interactive reasoning process.

this synergistic workflow, **MARS-SQL** achieves strong empirical results, reaching a new state-of-the-art Execution Accuracy of 77.84% on the BIRD development set and 89.75% on the Spider test set. Even when compared through efficiency-oriented analyses under matched latency budgets (Appendix F), **MARS-SQL** outperforms strictly prompted or self-consistency baselines, supporting the view that its gains are not explained by added parameters or inference time alone.

Our main contributions are as follows:

1. We introduce **MARS-SQL**, a trainable multi-agent framework for Text-to-SQL. Our contribution is primarily at the workflow level: we separate schema grounding, interactive generation, and outcome validation, and train these roles instead of relying on a static prompted pipeline.
2. We show how to train an **interactive multi-turn generation policy** with execution feedback. The Generation Agent operates within a Think-Act-Observe loop and learns to diagnose errors and self-correct from live database feedback, while the Grounding Agent further improves schema context management through data-driven training.
3. **MARS-SQL** achieves strong empirical results, reaching 77.84% execution accuracy on BIRD-dev and 89.75% on Spider-test, while also generalizing well to additional out-of-domain benchmarks. We further

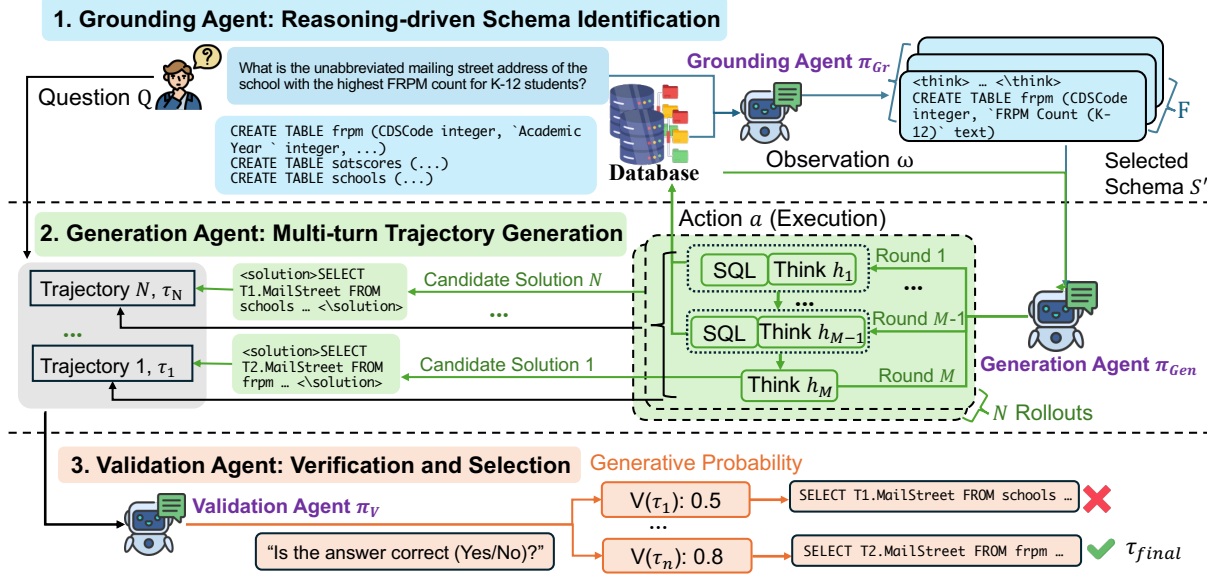


Figure 2. The three-stage workflow of MARS-SQL. (1) Grounding: A Grounding Agent selects the relevant schema. (2) Generation: A Generation agent produces multiple interaction trajectories using a Think-Act-Observe loop. (3) Validation: A Validation agent scores and selects the best trajectory.

show that the gains are not reducible to raw parameter scaling alone, and provide an efficiency analysis that characterizes the accuracy–latency–memory trade-off of the full framework rather than evaluating it only as a single operating point.

2. Preliminaries

Background Formulation. The primary goal of a Text-to-SQL system is to translate a natural language question into an executable SQL query. We can formally define this task as learning a mapping from a user question and a group of database schemas to the corresponding SQL query.

Let Q be the natural language question posed by a user. Let S be the database schema, which defines the structure of the database. The schema S consists of a set of tables $T = \{t_1, t_2, \dots, t_m\}$, where each table t_i is composed of a set of columns $C_i = \{c_{i,1}, c_{i,2}, \dots, c_{i,k}\}$. The schema also includes information about data types, primary keys (PKs), and foreign keys (FKs) that define the relationships between tables. The objective is to generate a SQL query Y such that when it is executed on the database instance D , it produces the correct answer to the question Q .

Conventionally, the Text-to-SQL problem is treated as a sequence-to-sequence translation task, where the goal is to learn a function $f: Y = f(Q, S)$. This formulation, however, treats the generation as a single, static step and fails to capture the exploratory and corrective nature required for solving complex analytical queries.

Reformulation as an Interactive Decision Process. As highlighted in the introduction, the static, one-shot formulation is insufficient for complex reasoning. A human analyst does not simply translate; they interact, explore, and refine. To model this more robust process, we reformulate Text-to-SQL as a sequential decision-making task, grounded in the ReAct paradigm (Yao et al., 2022a).

Instead of learning a direct mapping to a final query, our goal is to learn an optimal **policy**, π , that generates a **trajectory** of thoughts and actions to solve the problem. A complete interaction trajectory, τ , is a sequence of multiple rounds:

$$\tau = (h_1, \alpha_1, \omega_1, \dots, h_M, \alpha_M, \omega_M) \quad (1)$$

Each turn in the trajectory consists of:

- **Thought (h_t):** An internal reasoning step where the agent analyzes the problem state, reflects on past observations, and plans the next action.
- **Action (α_t):** An operation chosen by the agent from a predefined action space $\mathcal{A}$, primarily involving the execution of SQL queries against the database.
- **Observation (ω_t):** The feedback received from the environment after executing action α_t . This could be a query result, a database error, or other information that guides the agent’s next thought.

Under this formulation, the objective is to learn an optimal policy $\pi(\alpha_t|Q, S, (h_{<t}, \alpha_{<t}, \omega_{<t}))$ that maximizes the expected total reward over the trajectory, $E[R(\tau)]$. The reward

$R(\tau)$ is typically determined by the final outcome, specifically, whether the trajectory successfully produces a correct SQL query. This interactive, policy-based formulation naturally accommodates the trial-and-error and self-correction that are essential for tackling complex, real-world database queries.

3. Methodology

As illustrated in Figure 2, we introduce **MARS-SQL**, a trainable multi-agent framework that treats Text-to-SQL generation as an interactive, tool-augmented decision-making process. Our goal is not to propose a new standalone SQL primitive, but to make role decomposition and agent–database interaction trainable. The framework operates in three stages: Grounding, Generation, and Validation. Initially, a Grounding Agent prunes the full database schema to only the tables and columns relevant to the user question. Subsequently, a Generation agent executes a multi-turn rollout, producing multiple distinct interaction trajectories by actively querying the database. Finally, a Validation Agent scores each trajectory, and the one with the highest confidence score is selected as the final answer.

3.1. Grounding Agent: Reasoning-driven Schema Identification

The Grounding Agent performs table-level schema linking. Its goal is to learn a policy π_{Ground} . For each table $t_i \in T (1 \leq i \leq F)$ and the user’s question Q , the agent takes the pair $x_i = (Q, t_i)$ as input. It then generates a structured output $o_i = (d_i, C'_i)$, where $d_i \in \{\text{'Y'}, \text{'N'}\}$ is the relevance decision and $C'_i \subseteq C_i$ is the predicted subset of essential columns. The final output of this stage is the reduced schema S' , containing only the tables and columns deemed relevant: $S' = \{(t_i, C'_i) \mid o_i \text{ has } d_i = \text{'Y'}\}$.

Training Algorithm. We train the agent using **Group Relative Policy Optimization (GRPO)** (Shao et al., 2024). For each input x_i , the model generates a group of G candidate outputs $\{o_1, \dots, o_G\}$. The policy π_θ is then updated via the objective:

$$J_{GRPO}(\theta) = \mathbb{E} \left[\frac{1}{G} \sum_{j=1}^G \min \left(\frac{\pi_\theta(o_j|x_i)}{\pi_{\theta_{old}}(o_j|x_i)} A_j, \right. \right. \\ \left. \left. \text{clip} \left(\frac{\pi_\theta(o_j|x_i)}{\pi_{\theta_{old}}(o_j|x_i)}, 1 - \epsilon, 1 + \epsilon \right) A_j \right) - \beta D_{KL}(\pi_\theta \parallel \pi_{ref}) \right] \quad (2)$$

where A_j is the advantage for candidate o_j . The agent’s prompt template is in Appendix 26.

Reward Design. The reward function R_{Ground} provides a granular score based on the accuracy of the agent’s prediction. Let the agent’s parsed prediction be $P = (d_p, C_p)$,

where $d_p \in \{\text{'Y'}, \text{'N'}\}$ is the relevance decision and C_p is the set of predicted columns. Let the ground truth be $o^* = (d_g, C_g)$. The reward $R_g(o, o^*)$ is defined as:

$$R = \begin{cases} 1.0 & \text{if } o = o^* \\ \max(0.5, \frac{|C_g|}{|C_p|}) & \text{if } d_p = d_g = \text{'Y'} \text{ and } C_g \subset C_p \\ 0.2 & \text{if } d_p = \text{'Y'} \text{ and } d_g = \text{'N'} \\ 0.1 & \text{if } d_p = d_g = \text{'Y'} \text{ and } C_g \not\subseteq C_p \\ 0.0 & \text{if response format is invalid} \end{cases}$$

This heuristic reward shaping is grounded in the principle that missing schema elements (False Negatives) are fatal, while including extra ones (False Positives) is merely noisy.

3.2. Generation Agent: Multi-turn Trajectory Generation

The Generation Agent is the central component, tasked with producing SQL queries. Its **input** is the user question Q and the reduced schema S' from the Grounding Agent. Its **output** is a set of N candidate interaction trajectories, $\{\tau_1, \dots, \tau_N\}$, where each trajectory comprises of M rounds of the Think-Act-Observe process. The correct trajectory is expected to result in the final SQL solution Y_i .

MDP Formulation. We model the multi-turn generation process as a Markov Decision Process (MDP), defined by the tuple $(S, \mathcal{A}, P, R)$.

- **State Space S :** A state s_t represents the history of interaction up to round t , containing the sequence of past thoughts, actions, and observations $((h_1, \alpha_1, \omega_1), \dots, (h_{t-1}, \alpha_{t-1}, \omega_{t-1}))$.
- **Action Space $\mathcal{A}$:** An action $a_t = (h_t, \alpha_t)$ consists of generating a thought h_t and an executable SQL α_t .
- **Transition P :** $P(s_{t+1}|s_t, a_t)$ is the transition probability, which is determined by the environment (i.e., the database executing the action α_t).
- **Reward R :** The reward function $R_{gen}(\tau)$ provides a sparse signal based on the final outcome of a complete trajectory τ .

The goal is to learn a policy $\pi_{Gen}(a_t|s_t)$ that maximizes the return $J(\pi_{Gen}) = \mathbb{E}_{\tau \sim \pi_{Gen}}[R_{Gen}(\tau)]$.

Training. We train the policy π_{Gen} using Group Relative Policy Optimization (GRPO). For an input (Q, S') , we generate a group of G trajectories $\{\tau_1, \dots, \tau_G\}$, where each trajectory τ_{au_i} consists of a sequence of states and actions $(s_0^i, a_0^i, s_1^i, \dots)$. The GRPO objective for trajectories is de-

defined as formula 3,

$$J_{\text{GRPO}}(\theta) = \mathbb{E}_{\substack{(Q, S') \sim \mathcal{D}, \\ \{\tau_i\}_{i=1}^G \sim \pi_{\theta, \text{old}}}} \left[\frac{1}{G} \sum_{i=1}^G \sum_{t=0}^{|\tau_i|-1} \sum_{j=1}^{|a_t^i|} \min \left(\frac{\pi_{\theta}(a_{t,j}^i | s_t^i, a_{t,<j}^i)}{\pi_{\theta, \text{old}}(a_{t,j}^i | s_t^i, a_{t,<j}^i)} A_i, \right. \right. \\ \left. \left. \text{clip} \left(\frac{\pi_{\theta}(a_{t,j}^i | s_t^i, a_{t,<j}^i)}{\pi_{\theta, \text{old}}(a_{t,j}^i | s_t^i, a_{t,<j}^i)}, 1 - \epsilon, 1 + \epsilon \right) A_i \right) \right] \quad (3)$$

where $a_{t,j}^i$ is the j -th token of action a_t^i in trajectory τ_i , and A_i is the advantage for the entire trajectory, computed based on the relative rewards of all trajectories within the group. The reward signal $R_{\text{gen}}(\tau)$ used to compute A_i is derived solely from execution outcomes, encouraging the agent to prioritize both syntactic validity and semantic correctness:

$$R_{\text{gen}}(\tau) = \begin{cases} 1.0 & \text{if valid and execution correct} \\ 0.0 & \text{if valid but incorrect} \\ -1.0 & \text{if invalid} \end{cases}$$

This coarse but decisive feedback gives the agent freedom to discover effective reasoning strategies without being constrained to annotated step-level traces. Unlike Supervised Fine-Tuning (SFT) which typically relies on expensive, human-annotated Chain-of-Thought data, our sparse reward mechanism allows the agent to learn directly from environmental feedback. This enables the agent to autonomously discover diverse and robust error-recovery strategies that are often absent in static training sets. Consequently, MARS-SQL demonstrates superior data efficiency, achieving state-of-the-art results with only $\sim 35\text{k}$ training instances, whereas SFT baselines often require millions of synthetic examples to adequately cover the error space (detailed in Appendix E).

Interactive Reasoning. The agent is grounded in the ReAct paradigm, interleaving reasoning and acting in a Think-Act-Observe loop. This iterative structure transforms SQL generation from a one-shot translation into a dialogue with the database, enabling robust recovery from errors.

3.3. Validation Agent: Verification and Selection

The Validation Agent selects the optimal solution from the multiple candidates generated. Its **input** is the set of N candidate trajectories $\{\tau_1, \dots, \tau_N\}$ and the original question Q . Its **output** is the single best trajectory, τ_{final} . We employ a Generative Validation agent V , reframing verification as a next-token prediction task that leverages the base model’s own capabilities.

Training and Inference. The Validation Agent is trained via supervised fine-tuning (SFT) to generate a single token response: “Yes” for a correct trajectory or “No” for an incorrect one, conditioned on the question and trajectory. The prompt structure is in Appendix C.1.

At inference time, the agent’s score for a trajectory τ_i is the average probability of the “Yes” token across M stochastic

reasoning rounds :

$$V(\tau_i) = \frac{1}{M} \sum_{j=1}^M P(y_j = \text{“Yes”} | \tau_i, Q) \quad (4)$$

The trajectory with the highest confidence score is selected as the final answer. This method effectively turns the generative model into a high-quality reranker, capable of discerning the most plausible and accurate reasoning path among many alternatives.

4. Experiment

4.1. Experiment Setup

Implementations & Efficiency

All agents are based on Qwen2.5-Coder-7B-Instruct, implemented in PyTorch and trained on NVIDIA H800 GPUs. The Grounding and Generation Agents utilize RL via the Verl (Sheng et al., 2024) and SkyRL (Cao et al., 2025) frameworks, respectively, while the Validation Agent is fine-tuned (SFT) on agent-generated trajectories. For inference, we adopt $G = 8$ rollouts and $M = 8$ verification rounds. This is our performance-oriented configuration and delivers the best overall accuracy. Since the deployed system contains three specialized 7B agents, we explicitly report both the per-agent size (7B) and the total inference-time footprint ($\sim 21\text{B}$). We analyze its latency, token cost, VRAM footprint, normalized-budget comparisons, and alternative lower-cost operating points in Appendix F. The entire multi-agent training pipeline requires only ~ 13 hours (Appendix E). Detailed prompts and hyperparameters are provided in Appendix M, H and B.

Benchmark Dataset

All experiments are conducted on the BIRD (Li et al., 2023), Spider 1.0 (Yu et al., 2018) and Spider-DK (Gan et al., 2021) datasets. We use BIRD for in-domain evaluation and Spider and Spider-DK for out-of-domain evaluation; we further report SPaRC results in Appendix D. Dataset details are in Appendix C.2.

Evaluation Metric

We evaluate model performance using Execution Accuracy (EX), which is the primary metric for correctness. A predicted SQL query receives a score of 1 if its execution result is identical to the execution result of the ground-truth query, and 0 otherwise. The final score is the percentage of correctly executed queries.

Baseline models

To contextualize the performance of MARS-SQL, we conduct a comprehensive comparison against a diverse set of models. These are organized into three distinct categories:

Table 1. Main results on the BIRD-dev, Spider-test, and Spider-DK benchmarks. We report Execution Accuracy (%). ‘Thinking?’ indicates whether the method uses a multi-step reasoning process. For multi-agent systems, the size column reports the per-agent model size; **MARS-SQL** uses three 7B agents (~21B total at inference). **Bold** indicates the best result, and underline indicates the second best.

Model	Size	# agents	Thinking?	Training set	Bird-dev (%)	Spider-test (%)	Spider-DK (%)
<i>Base Models</i>							
O3-mini	-	-	Yes	-	61.34	78.82	71.77
Qwen-2.5-coder	7B	-	No	-	54.56	75.87	61.31
Qwen3-coder	30B	-	No	-	59.00	79.97	68.79
GPT-4o	-	-	No	-	61.90	77.10	72.9
GPT-5	-	-	No	-	65.45	78.39	66.92
<i>Closed-source Multi agentic framework</i>							
CHESS	-	4	No	-	65.00	87.2	-
OpenSearch-SQL+ GPT-4o	-	4	No	-	69.30	87.1	-
XiYan-SQL	-	3	No	-	73.34	<u>89.65</u>	-
CHASE-SQL + Gemini	-	4	Yes	-	<u>74.90</u>	87.6	-
<i>Open Source Agentic Framework</i>							
CodeS	7B	1	No	Spider	57.17	80.3	72.0
Share	8B	1	No	Bird	64.14	85.90	75.3
OmniSQL	32B	1	No	OmniSQL	64.5	87.60	76.1
Arctic-Text2SQL-R1	32B	1	Yes	Bird+Spider	70.70	88.80	81.5
Reasoning SQL	14B	1	Yes	Bird	72.29	81.43	73.03
MARS-SQL	7B each	3	Yes	Bird	77.84	89.75	<u>78.13</u>

Base models, High-performing closed-source systems, and Trained open-source models.

Base Models: This category includes foundational LLMs used without task-specific fine-tuning to establish a performance baseline. We evaluate O3-mini, GPT-4o (OpenAI, 2023), GPT-5 (Singh et al., 2025), Qwen2.5-coder-7B (Hui et al., 2024) and Qwen3-coder-30B (Yang et al., 2025). These results help gauge the inherent Text-to-SQL capabilities of modern LLMs before specialized training.

Closed Source Multi agentic framework: This category consists of systems that leverage powerful proprietary models via APIs, representing the upper bound of performance achievable with leading commercial technology. These methods, such as CHESS (Talei et al., 2024), OpenSearch-SQL (Xie et al., 2025b), XiYan-SQL (Gao et al., 2024), and CHASE-SQL (Pourreza et al., 2025a), typically employ sophisticated frameworks and prompting techniques. This comparison situates our open-source multi-agent framework performance against industry-leading systems.

Open Source Agentic Framework: This group comprises leading open-source models specifically fine-tuned for the Text-to-SQL task, representing the current state-of-the-art in the research community. These models, including CodeS (Li et al., 2024b), Share (Qu et al., 2025), OmniSQL (Li et al., 2025a), Arctic-Text2SQL-R1 (Yao et al., 2025), and Reasoning SQL (Pourreza et al., 2025b), employ various advanced training methodologies. Comparing **MARS-SQL** against these systems assesses its competitiveness and advancements over existing specialized methods.

4.2. Main Results

As presented in Table 1, our method, **MARS-SQL**, trained solely on the BIRD training set, achieves state-of-the-art execution accuracy on both the Bird-dev (77.84%) and the Spider-test (89.75%). Additionally, it obtains the second-highest score on the Spider-DK benchmark with 78.13%.

In-Domain Performance on BIRD-dev.

On the in-domain BIRD-dev set, **MARS-SQL** establishes a new state-of-the-art with an execution accuracy of **77.84%**. This result represents a significant improvement of 5.55% over the next best open-source competitor, Reasoning SQL (72.29%). More impressively, our system outperforms all listed closed-source solutions, including the strong CHASE-SQL + Gemini (74.90%). This demonstrates the superior effectiveness of our training methodology on this complex, real-world benchmark.

Out-of-Domain Generalization.

The out-of-domain generalization of **MARS-SQL** is particularly noteworthy, demonstrated by its strong performance on both the Spider-test and Spider-DK benchmarks. On the broad Spider-test set, it achieves a state-of-the-art score of **89.75%**, showcasing exceptional generalization to unseen schemas and question types. This robustness extends to the specialized Spider-DK benchmark, which tests for implicit domain knowledge, where **MARS-SQL** secures a competitive second-highest score of **78.13%**. Crucially, these results were achieved without any exposure to the Spider training

Table 2. Ablation study on the components of our multi-agent framework. We evaluate the contribution of each agent (Grounder, Verifier). The final row, **MARS-SQL**, represents our full proposed model.

Configuration	Agent Size	# agents	Bird dev (%)	Spider test (%)	Spider DK (%)
Generator Only (Base)	7B	1	66.37	80.11	69.91
w/o verifier (Grounder+ RL Generator)	7B	2	68.71	80.72	70.65
w/o Grounder (RL Generator + Verifier)	7B	2	69.75	89.19	77.01
w/ Self-Consistency (instead of Verifier)	7B	2	72.93	83.51	73.08
MARS-SQL (Full Framework)	7B	3	77.84	89.75	78.13

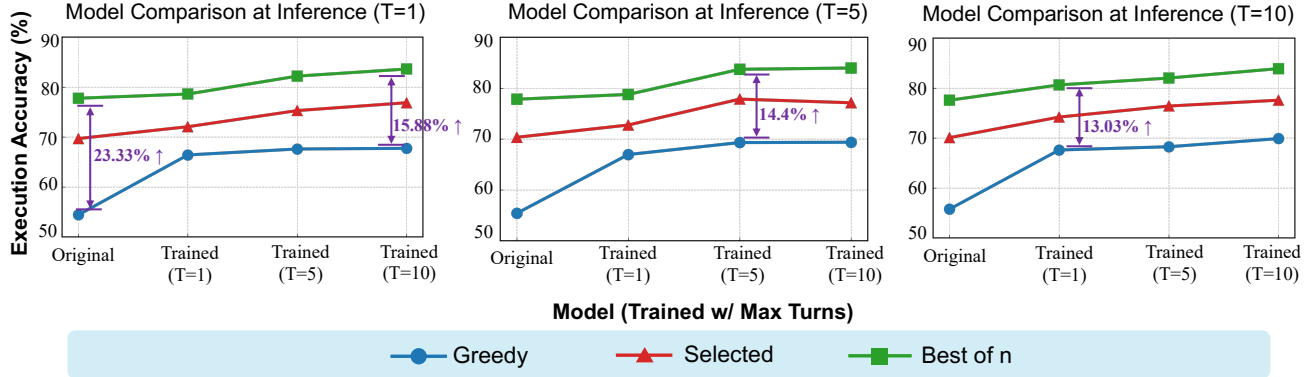


Figure 3. Execution accuracy on Bird-dev of models fine-tuned with different maximum interaction turns (T), evaluated at inference turn limits of 1, 5, and 10. ‘**Greedy**’ uses a single generation trajectory ($N = 1$) without validation; ‘**Selected**’ denotes the final trajectory chosen by the Validation Agent from $N = 8$ candidates; and ‘**Best of N**’ represents the oracle upper bound where the question is considered correct if any of the N candidates matches.

set. This contrasts with competitors like Arctic-Text2SQL-R1, which required training on Spider data (from which Spider-DK is derived) to achieve its high scores. We further include additional out-of-domain evidence on SPaRC in Appendix D. Together, these results highlight that training solely on the diverse BIRD dataset effectively equips the model for broad cross-domain and interaction-heavy challenges.

4.3. Ablation Studies

Multi-agent frame components analysis.

We conduct a systematic ablation study to validate the contribution of each key component in our **MARS-SQL** framework, with results presented in Table 2. The analysis confirms that both the Schema Grounder and the Generative Validation Agent are useful components, though their impact is not uniform across datasets. The Validation Agent provides the most consistent gain: our purpose-built verifier substantially outperforms a strong alternative like Self-Consistency (77.84% vs. 72.93% on BIRD-dev), highlighting the benefits of specialized validation. The Grounder is especially helpful on BIRD-dev and more modest on Spider-test, which is consistent with our later analysis that grounding matters most under larger, noisier, or more ambiguous schemas. We therefore view the framework’s gain

as arising from complementary roles rather than from every component contributing equally in every setting. This interpretation is further reinforced by single-model multi-task baselines: when a single Qwen2.5-Coder model is SFT-trained to perform grounding, generation, and validation within one shared model, 14B and 32B variants reach only 55.67% and 57.82% EX on BIRD-dev, respectively (Appendix G).

Controlled isolation of component gains.

To better separate the effects of structured inference, learned interaction, and reranking, we further conduct a dataset-wide controlled-isolation analysis on BIRD-dev. Starting from a strict baseline consisting of the RL Generator only with $T=1$ and greedy decoding, we add exactly one mechanism at a time using configurations already studied in our manuscript. The results are summarized in Table 3, with further details in Appendix J.

This analysis sharpens the causal picture in two ways. First, the largest isolated gain comes from generative verification, which is consistent with our ablations and selection-method comparisons. Second, the RL interactive policy still contributes 44 additional correct queries under greedy decoding alone, without relying on sampling or reranking. This provides direct evidence that execution-feedback-driven interaction improves the underlying policy itself rather than

Table 3. Dataset-wide controlled isolation on BIRD-dev. We report the additional queries resolved beyond the strict RL-Generator baseline (62.78% EX).

Mechanism	Extra Solved	Gain
Schema pruning	56	+3.63%
RL interactive reasoning	44	+2.89%
Generative verification	131	+8.54%
Total improvement	231	+15.06%

only helping through best-of- N selection.

Influence of different max interaction turns.

We then study the impact of the maximum interaction turns (T) during Reinforcement Learning. The results are visualized in Figure 3, with full details provided in Appendix K. As shown, increasing T from 1 to 10 consistently improves both Greedy and Best of 8 accuracy. Notably, our model trained with $T=10$ significantly outperforms models trained with fewer turns across all inference settings. For instance, at Inference ($T=10$), it achieves 69.88% Greedy accuracy, surpassing the $T=1$ model (67.60%) and the base model (55.76%). Furthermore, this process enhances single-pass reliability by narrowing the gap between Best of 8 (potential) and Greedy (actual) performance. This gap shrinks from a substantial 23.33% in the base model to 12.19% in the $T=1$ model at Inference ($T=1$). Training with a larger T reinforces this effect, making the model’s greedy output more aligned with its optimal potential, thereby improving its dependability. Beyond accuracy, RL also improves interaction efficiency: the trained Generation Agent uses fewer average interaction turns than the base model across difficulty levels, while still retaining the ability to spend more turns on long-tail difficult cases. We report this rollout-length evolution and turn distribution in Appendix H, Table 21, and Table 22.

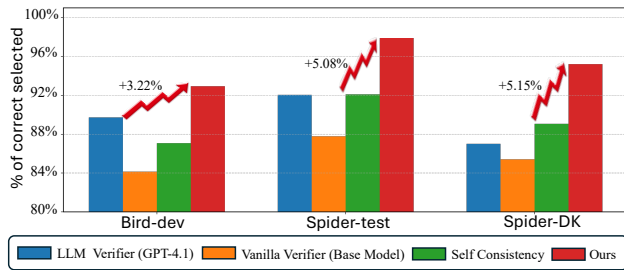


Figure 4. Comparison of different selection strategies.

Selection methods analysis.

To validate the effectiveness of our Generative Validation Agent, we compare it against several alternative selection strategies, as illustrated in Figure 4. While common approaches such as Self-Consistency or using a powerful LLM as a Judge (e.g., GPT-4.1) provide a reasonable baseline,

their performance is suboptimal and inconsistent across the different benchmarks. In contrast, our fine-tuned Generative Validation Agent consistently outperforms all other methods by a significant margin. On the challenging Spider-test, it achieves a correct selection rate of 97.15%, a substantial improvement over the next-best strategy’s 92.09%. Similar significant gains are observed on both BIRD-dev and Spider-DK. This consistent superiority demonstrates the stability and robustness of our specialized approach. Unlike general-purpose models or heuristic-based methods, our Validation Agent reliably identifies the most accurate reasoning trajectory, making it a critical component for achieving state-of-the-art performance. Full execution accuracy results are detailed in Appendix Q.

5. Related Work

LLMs for Text-to-SQL

The rise of Large Language Models (LLMs) has brought notable progress to Text-to-SQL tasks, moving past traditional sequence-to-sequence approaches. Recent studies emphasize in-context learning, where strategies such as Chain-of-Thought (CoT) prompting are used to break down complex queries into intermediate reasoning steps (Tai et al., 2023; Dong et al., 2023). Frameworks like DIN-SQL (Pourreza & Rafiei, 2023) and DAIL-SQL (Gao et al., 2023) have systematically explored prompt engineering and multi-stage pipelines that include schema linking, generation, and refinement to boost performance. Building on these ideas, more recent studies (Wang et al., 2025a; Deng et al., 2025; Gao et al., 2024; Xie et al., 2025b; Guo et al., 2025b) move toward structured, multi-step workflows that better match the complexity of real databases and diverse queries. Notably, while works like MSc-SQL (Gorti et al., 2025) employ multi-sample critiquing via supervised fine-tuning, our work shifts away from fixed datasets or static prompting by introducing a dynamic, reinforcement-learning-based agentic system that optimizes interaction trajectories directly against execution feedback.

Multi-Agent systems

Large Language Models (LLMs) have enabled sophisticated multi-agent systems by adopting specialized roles via in-context prompting (Wang et al., 2024; Min et al., 2022). Our focus is on goal-oriented problem-solving frameworks, rather than social simulations (Zhang et al., 2024; Hua et al., 2023; Yuan et al., 2023), where tasks are divided among collaborating agents. The complexity of these collaborations has grown from simple debating (Du et al., 2024) to structured workflows with the use of tools, such as software development agents ChatDev (Qian et al., 2024), MedTutor-R1 (He et al., 2025b) and CollabUIAgent (He et al., 2025a). Other notable approaches include the generic framework

AutoGen (Wu et al., 2024) and the dynamic cooperation in AutoAgents (Chen et al., 2023). Following this established paradigm, we propose a specialized pipeline for Text-to-SQL using Grounding, Generation, and Validation agents.

Reinforcement Learning

Reinforcement Learning (RL) is increasingly used to enhance the complex reasoning capabilities of LLMs, especially when combined with chain-of-thought prompting (Wei et al., 2022; Jaech et al., 2024; Su et al., 2025). This approach has proven highly effective, achieving state-of-the-art results in fields like mathematics and code generation (Qin et al., 2024; Zhao et al., 2024). Typical approaches fine-tune models with policy gradient methods such as PPO or GRPO, rewarding logical soundness or correct outcomes (Shao et al., 2024; Guo et al., 2025a; Huang et al., 2025b; He et al., 2025b). While PPO is a common choice, GRPO offers advantages by being less prone to high variance and more memory-efficient, as it does not require loading an additional critic model. In parallel, interactive reasoning paradigms like ReAct (Yao et al., 2022b) leverage prompting-based Think–Act–Observe loops to enable tool use and self-correction, but without explicit policy training. While Text-to-SQL requires similarly complex reasoning, explicit RL for this domain remains underexplored. Our work addresses this gap by training the Generation agent’s policy with execution-based rewards, enabling robust, stateful query generation and dynamic self-correction.

Test-Time Scaling

To enhance performance without the cost of retraining, many researchers have focused on inference-time techniques. Self-consistency, for instance, has become a popular method where multiple reasoning paths are sampled and the final answer is chosen by majority vote (Wang et al., 2022). This concept has been further refined by verification and reranking methods, which employ an external mechanism or model to score and select the best candidate from a pool of outputs (Zheng et al., 2023; Gu et al., 2024). Our approach builds on the recent innovation of Generative Verifiers (Zhang et al., 2025b). Instead of a voting process or a separate classifier, our Validation Agent reframes selection as a next-token prediction problem. It assesses each potential solution trajectory by calculating the probability of the model generating a “Yes” token, ultimately selecting the trajectory with the highest confidence score.

6. Conclusion

In this work, we introduced **MARS-SQL**, the first framework to leverage an **interactive RL policy for tool-augmented SQL generation under sparse execution rewards** within a multi-agent decomposition. Unlike static, single-pass approaches, our method uniquely couples this

dynamic generation policy, which learns to reason and self-correct via a ReAct-style loop, with a **generative probability-based trajectory selection** mechanism tailored for SQL verification. This unified methodology enables **MARS-SQL** to establish new state-of-the-art execution accuracies on BIRD (77.84%) and Spider (89.75%), demonstrating exceptional cross-domain generalization by achieving top-tier performance on Spider without any in-domain training. Our ablation studies confirm the criticality of this synergistic design, suggesting that shifting from prompt engineering to interactive, policy-driven reasoning is a pivotal direction for building dependable, data-centric AI systems.

At the same time, we emphasize an important practical limitation. Although each module is 7B-scale, the deployed system contains three specialized agents and therefore has an inference-time footprint of approximately 21B parameters. This introduces a real trade-off between multi-agent specialization and single-model scaling. Our new analyses show that the observed accuracy gains are not explained by parameter count alone, but the deployment cost of the full pipeline remains higher than that of a single 7B model. We therefore view transparent reporting of total parameters, latency, token usage, and VRAM as essential for future work on efficient Text-to-SQL agents.

Impact Statement

This paper introduces MARS-SQL, a multi-agent framework designed to advance the field of Machine Learning by enabling more robust and self-correcting autonomous agents in complex data environments. By reframing Text-to-SQL as an interactive decision-making process grounded in reinforcement learning, our work significantly lowers the technical barriers for non-expert users to interact with large-scale, professional databases, thereby fostering the democratization of data-driven insights across diverse sectors. The specialized roles of grounding, generation, and validation agents directly address the critical societal need for more dependable AI systems, specifically by mitigating common risks such as model hallucinations and the propagation of incorrect query results in real-world applications. While this technology empowers individuals to perform complex data analysis without advanced programming knowledge, it also emphasizes a move toward transparent and verifiable AI, encouraging a deployment model that balances automation with human-in-the-loop oversight to ensure the ethical and responsible use of generated information.

Acknowledgements

This research was financially supported by the Open Research Fund from Guangdong Laboratory of Artificial Intelligence and Digital Economy (SZ), under Grant No. GML-KF-26-06. We also thank the anonymous reviewers for their constructive feedback.

References

- Cao, S., Li, D., Zhao, F., Yuan, S., Hegde, S. R., Chen, C., Ruan, C., Griggs, T., Liu, S., Tang, E., Liaw, R., Moritz, P., Zaharia, M., Gonzalez, J. E., and Stoica, I. Skyrl-agent: Efficient rl training for multi-turn llm agent. *arXiv preprint arXiv:2511.16108*, 2025.
- Chang, C.-Y., Jiang, Z., Rakesh, V., Pan, M., Yeh, C.-C. M., Wang, G., Hu, M., Xu, Z., Zheng, Y., Das, M., et al. Main-rag: Multi-agent filtering retrieval-augmented generation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 2607–2622, 2025.
- Chaturvedi, S., Chadha, A., and Bindschaedler, L. Sql-of-thought: Multi-agentic text-to-sql with guided error correction. *arXiv preprint arXiv:2509.00581*, 2025.
- Chen, G., Dong, S., Shu, Y., Zhang, G., Sesay, J., Karlsson, B. F., Fu, J., and Shi, Y. Autoagents: A framework for automatic agent generation. *arXiv preprint arXiv:2309.17288*, 2023.
- Deng, M., Ramachandran, A., Xu, C., Hu, L., Yao, Z., Datta, A., and Zhang, H. Reforce: a text-to-sql agent with self-refinement, consensus enforcement, and column exploration. *arXiv preprint arXiv:2502.00675*, 2025.
- Dong, X., Zhang, C., Ge, Y., Mao, Y., Gao, Y., Chen, L., Lin, J., and Lou, D. C3: Zero-shot text-to-sql with chatgpt. *arXiv preprint arXiv:2307.07306*, 2023.
- Du, Y., Li, S., Torralba, A., Tenenbaum, J. B., and Mordatch, I. Improving factuality and reasoning in language models through multiagent debate. In *Forty-first international conference on machine learning*, 2024.
- Gan, Y., Chen, X., and Purver, M. Exploring underexplored limitations of cross-domain text-to-sql generalization. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp. 8926–8931, 2021.
- Gao, D., Wang, H., Li, Y., Sun, X., Qian, Y., Ding, B., and Zhou, J. Text-to-sql empowered by large language models: A benchmark evaluation. *arXiv preprint arXiv:2308.15363*, 2023.
- Gao, Y., Liu, Y., Li, X., Shi, X., Zhu, Y., Wang, Y., Li, S., Li, W., Hong, Y., Luo, Z., et al. A preview of xiyansql: A multi-generator ensemble framework for text-to-sql. *arXiv preprint arXiv:2411.08599*, 2024.
- Gorti, S. K., Gofman, I., Liu, Z., Wu, J., Vouitsis, N., Yu, G., Cresswell, J. C., and Hosseinzadeh, R. Msc-sql: Multi-sample critiquing small language models for text-to-sql translation. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 2145–2160, 2025.
- Gu, J., Jiang, X., Shi, Z., Tan, H., Zhai, X., Xu, C., Li, W., Shen, Y., Ma, S., Liu, H., et al. A survey on llm-as-a-judge. *The Innovation*, 2024.
- Gui, L., Gârbaacea, C., and Veitch, V. Bonbon alignment for large language models and the sweetness of best-of-n sampling. *Advances in Neural Information Processing Systems*, 37:2851–2885, 2024.
- Guo, D., Yang, D., Zhang, H., Song, J., Wang, P., Zhu, Q., Xu, R., Zhang, R., Ma, S., Bi, X., et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025a.
- Guo, T., Wang, H., Liu, C., Golalikhani, M., Chen, X., Zhang, X., and Reddy, C. K. Mtsql-r1: Towards long-horizon multi-turn text-to-sql via agentic training. *arXiv preprint arXiv:2510.12831*, 2025b.
- He, Z., Liu, Z., Li, P., Fung, Y. R., Yan, M., Zhang, J., Huang, F., and Liu, Y. Advancing language multi-agent

- learning with credit re-assignment for interactive environment generalization. *arXiv preprint arXiv:2502.14496*, 2025a.
- He, Z., Yang, H., Qin, Z., and Fung, Y. R. Medtutor-r1: Socratic personalized medical teaching with multi-agent simulation. *arXiv preprint arXiv:2512.05671*, 2025b.
- He, Z., Yang, H., Min, R., Qin, Z., and Fung, Y. R. On stable long-form generation: Benchmarking and mitigating length volatility. *arXiv preprint arXiv:2605.01357*, 2026.
- Hong, Z., Yuan, Z., Zhang, Q., Chen, H., Dong, J., Huang, F., and Huang, X. Next-generation database interfaces: A survey of llm-based text-to-sql. *IEEE Transactions on Knowledge and Data Engineering*, 2025.
- Hua, W., Fan, L., Li, L., Mei, K., Ji, J., Ge, Y., Hemphill, L., and Zhang, Y. War and peace (waragent): Large language model-based multi-agent simulation of world wars. *arXiv preprint arXiv:2311.17227*, 2023.
- Huang, J., He, Z., Huang, Y., Polisetty, S., Wang, Q., and Fung, Y. R. Mac-tuning: Llm multi-compositional problem reasoning with enhanced knowledge boundary awareness. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 663–676, 2025a.
- Huang, S., Wang, H., Zhong, W., Su, Z., Feng, J., Cao, B., and Fung, Y. R. Adactrl: Towards adaptive and controllable reasoning via difficulty-aware budgeting. *arXiv preprint arXiv:2505.18822*, 2025b.
- Huang, Y., Li, S., Liu, M., Liu, W., Huang, S., Fan, Z., Chan, H. P., and Fung, Y. R. M. Scaling environments for llm agents in the era of learning from interaction: A survey. *arXiv preprint arXiv:2511.09586*, 2025c.
- Hui, B., Yang, J., Cui, Z., Yang, J., Liu, D., Zhang, L., Liu, T., Zhang, J., Yu, B., Lu, K., et al. Qwen2. 5-coder technical report. *arXiv preprint arXiv:2409.12186*, 2024.
- Jaech, A., Kalai, A., Lerer, A., Richardson, A., El-Kishky, A., Low, A., Helyar, A., Madry, A., Beutel, A., Carney, A., et al. Openai o1 system card. *arXiv preprint arXiv:2412.16720*, 2024.
- Lei, F., Chen, J., Ye, Y., Cao, R., Shin, D., Su, H., Suo, Z., Gao, H., Hu, W., Yin, P., et al. Spider 2.0: Evaluating language models on real-world enterprise text-to-sql workflows. In *International Conference on Learning Representations*, volume 2025, pp. 28691–28735, 2025.
- Li, B., Luo, Y., Chai, C., Li, G., and Tang, N. The dawn of natural language to sql: Are we fully ready? *arXiv preprint arXiv:2406.01265*, 2024a.
- Li, H., Zhang, J., Liu, H., Fan, J., Zhang, X., Zhu, J., Wei, R., Pan, H., Li, C., and Chen, H. Codes: Towards building open-source language models for text-to-sql. *Proceedings of the ACM on Management of Data*, 2(3):1–28, 2024b. doi: 10.1145/3654930.
- Li, H., Wu, S., Zhang, X., Huang, X., Zhang, J., Jiang, F., Wang, S., Zhang, T., Chen, J., Shi, R., Chen, H., and Li, C. Omnisql: Synthesizing high-quality text-to-sql data at scale. *arXiv preprint arXiv:2503.02240*, 2025a.
- Li, J., Hui, B., Qu, G., Yang, J., Li, B., Li, B., Wang, B., Qin, B., Geng, R., Huo, N., et al. Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls. *Advances in Neural Information Processing Systems*, 36:42330–42357, 2023.
- Li, J., Li, X., Qu, G., Jacobsson, P., Qin, B., Hui, B., Si, S., Huo, N., Xu, X., Zhang, Y., et al. Swe-sql: Illuminating llm pathways to solve user sql issues in real-world applications. *Advances in Neural Information Processing Systems*, 38:97085–97120, 2025b.
- Li, J., Wu, T., Mao, Y., Gao, Y., Feng, Y., and Liu, H. Sql-factory: a multi-agent framework for high-quality and large-scale sql generation. *Proceedings of the VLDB Endowment*, 19(3):292–305, 2025c.
- Li, Y., Choi, D., Chung, J., Kushman, N., Schrittwieser, J., Leblond, R., Eccles, T., Keeling, J., Gimeno, F., Dal Lago, A., et al. Competition-level code generation with alpha-code. *Science*, 378(6624):1092–1097, 2022.
- Liu, J., Qian, C., Su, Z., Zong, Q., Huang, S., He, B., and Fung, Y. R. Costbench: Evaluating multi-turn cost-optimal planning and adaptation in dynamic environments for llm tool-use agents. *arXiv preprint arXiv:2511.02734*, 2025.
- Ma, P., Zhuang, X., Xu, C., Jiang, X., Chen, R., and Guo, J. Sql-r1: Training natural language to sql reasoning model by reinforcement learning. *Advances in Neural Information Processing Systems*, 38:174505–174537, 2026.
- Min, S., Lyu, X., Holtzman, A., Artetxe, M., Lewis, M., Hajishirzi, H., and Zettlemoyer, L. Rethinking the role of demonstrations: What makes in-context learning work? In *Proceedings of the 2022 conference on empirical methods in natural language processing*, pp. 11048–11064, 2022.
- Ni, A., Iyer, S., Radev, D., Stoyanov, V., Yih, W.-t., Wang, S., and Lin, X. V. Lever: Learning to verify language-to-code generation with execution. In *International Conference on Machine Learning*, pp. 26106–26128. PMLR, 2023.
- OpenAI. Gpt-4 technical report, 2023.

- Pourreza, M. and Rafiei, D. Din-sql: Decomposed in-context learning of text-to-sql with self-correction. In *Advances in Neural Information Processing Systems*, volume 36, pp. 36339–36348, 2023.
- Pourreza, M., Li, H., Sun, R., Chung, Y., Talaei, S., Kakkar, G. T., Gan, Y., Saberi, A., Ozcan, F., and Arik, S. Chase-sql: Multi-path reasoning and preference optimized candidate selection in text-to-sql. In *International Conference on Learning Representations*, volume 2025, pp. 60385–60415, 2025a.
- Pourreza, M., Talaei, S., Sun, R., Wan, X., Li, H., Mirhoseini, A., Saberi, A., Arik, S., et al. Reasoning-sql: Reinforcement learning with sql tailored partial rewards for reasoning-enhanced text-to-sql. *arXiv preprint arXiv:2503.23157*, 2025b.
- Qian, C., Liu, W., Liu, H., Chen, N., Dang, Y., Li, J., Yang, C., Chen, W., Su, Y., Cong, X., et al. Chatdev: Communicative agents for software development. In *Proceedings of the 62nd annual meeting of the association for computational linguistics (volume 1: Long papers)*, pp. 15174–15186, 2024.
- Qin, Y., Liang, S., Ye, Y., Zhu, K., Yan, L., Lu, Y., Lin, Y., Cong, X., Tang, X., Qian, B., et al. Toolllm: Facilitating large language models to master 16000+ real-world apis. In *International Conference on Learning Representations*, volume 2024, pp. 9695–9717, 2024.
- Qu, G., Li, J., Qin, B., Li, X., Huo, N., Ma, C., and Cheng, R. SHARE: An SLM-based hierarchical action CorREction assistant for text-to-SQL. In Che, W., Nabende, J., Shutova, E., and Pilehvar, M. T. (eds.), *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 11268–11292, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.552.
- Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, H., Zhang, M., Li, Y., Wu, Y., et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Sheng, G., Zhang, C., Ye, Z., Wu, X., Zhang, W., Zhang, R., Peng, Y., Lin, H., and Wu, C. Hybridflow: A flexible and efficient rlhf framework. *arXiv preprint arXiv:2409.19256*, 2024.
- Singh, A., Fry, A., Perelman, A., Tart, A., Ganesh, A., El-Kishky, A., McLaughlin, A., Low, A., Ostrow, A., Ananthram, A., et al. Openai gpt-5 system card. *arXiv preprint arXiv:2601.03267*, 2025.
- Su, Z., Xia, P., Guo, H., Liu, Z., Ma, Y., Qu, X., Liu, J., Li, Y., Zeng, K., Yang, Z., et al. Thinking with images for multimodal reasoning: Foundations, methods, and future frontiers. *arXiv preprint arXiv:2506.23918*, 2025.
- Tai, C.-Y., Chen, Z., Zhang, T., Deng, X., and Sun, H. Exploring chain of thought style prompting for text-to-sql. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 5376–5393, 2023.
- Talaei, S., Pourreza, M., Chang, Y.-C., Mirhoseini, A., and Saberi, A. Chess: Contextual harnessing for efficient sql synthesis. *arXiv preprint arXiv:2405.16755*, 2024.
- Wang, B., Ren, C., Yang, J., Liang, X., Bai, J., Chai, L., Yan, Z., Zhang, Q.-W., Yin, D., Sun, X., and Li, Z. Mac-sql: A multi-agent collaborative framework for text-to-sql. In *Proceedings of the 31st International Conference on Computational Linguistics (COLING)*, pp. 540–557, Abu Dhabi, UAE, 2025a. Association for Computational Linguistics.
- Wang, B., Ren, C., Yang, J., Liang, X., Bai, J., Chai, L., Yan, Z., Zhang, Q.-W., Yin, D., Sun, X., et al. Mac-sql: A multi-agent collaborative framework for text-to-sql. In *Proceedings of the 31st International Conference on Computational Linguistics*, pp. 540–557, 2025b.
- Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., Chowdhery, A., and Zhou, D. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*, 2022.
- Wang, X., Xiao, Y., Huang, J.-t., Yuan, S., Xu, R., Guo, H., Tu, Q., Fei, Y., Leng, Z., Wang, W., et al. Incharacter: Evaluating personality fidelity in role-playing agents through psychological interviews. In *Proceedings of the 62nd annual meeting of the association for computational linguistics (volume 1: Long papers)*, pp. 1840–1873, 2024.
- Wang, X., Chen, W., Li, C., Zhang, C., Yang, R., Zhao, W. X., and Wen, J.-R. On the Resilience of LLM-Based Multi-Agent Collaboration with Faulty Agents. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 1124–1145, Vienna, Austria, July 2025c. Association for Computational Linguistics.
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q. V., Zhou, D., et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.

- Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., et al. Autogen: Enabling next-gen llm applications via multi-agent conversations. In *First conference on language modeling*, 2024.
- Xie, E., Xiong, G., Yang, H., Coleman, O., Kennedy, M., and Zhang, A. Leveraging grounded large language models to automate educational presentation generation. In *Large Foundation Models for Educational Assessment*, pp. 207–220. PMLR, 2025a.
- Xie, X., Xu, G., Zhao, L., and Guo, R. Opensearch-sql: Enhancing text-to-sql with dynamic few-shot and consistency alignment. *Proceedings of the ACM on Management of Data*, 3(3):1–24, 2025b.
- Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022a.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. R., and Cao, Y. React: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*, 2022b.
- Yao, Z., Sun, G., Borchmann, L., Nuti, G., Shen, Z., Deng, M., Zhai, B., Zhang, H., Li, A., and He, Y. Arctic-text2sql-r1: Simple rewards, strong reasoning in text-to-sql. *arXiv preprint arXiv:2505.20315*, 2025.
- Yu, T., Zhang, R., Yang, K., Yasunaga, M., Wang, D., Li, Z., Ma, J., Li, I., Yao, Q., Roman, S., et al. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task. In *Proceedings of the 2018 conference on empirical methods in natural language processing*, pp. 3911–3921, 2018.
- Yu, T., Zhang, R., Yasunaga, M., Wang, X., Li, I., Yang, Q., Wang, Z., Li, A., Radev, D., et al. Sparc: Cross-domain semantic parsing in context. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019.
- Yuan, L., Chen, Y., Wang, X., Fung, Y. R., Peng, H., and Ji, H. Craft: Customizing llms by creating and retrieving from specialized toolsets. *arXiv preprint arXiv:2309.17428*, 2023.
- Zhang, J., Yang, H., Miao, K., Zhang, R., Pi, R., Gao, J., and Zhou, X. Exesql: Self-taught text-to-sql models with execution-driven bootstrapping for sql dialects. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pp. 24305–24326, 2025a.
- Zhang, L., Hosseini, A., Bansal, H., Kazemi, S. M., Kumar, A., and Agarwal, R. Generative verifiers: Reward modeling as next-token prediction. In *International Conference on Learning Representations*, volume 2025, pp. 12476–12505, 2025b.
- Zhang, Y., Sun, R., Chen, Y., Pfister, T., Zhang, R., and Arik, S. Ö. Chain of agents: Large language models collaborating on long-context tasks. *Advances in Neural Information Processing Systems*, 37:132208–132237, 2024.
- Zhao, Y., Yin, H., Zeng, B., Wang, H., Shi, T., Lyu, C., Wang, L., Luo, W., and Zhang, K. Marco-o1: Towards open reasoning models for open-ended solutions. *arXiv preprint arXiv:2411.14405*, 2024.
- Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E., et al. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in neural information processing systems*, 36: 46595–46623, 2023.

A. The Use of Large Language Models

Large Language Models (LLMs) were utilized in a limited, assistive capacity for specific tasks in this project. For manuscript preparation, the authors supplied their own draft to an LLM, which then provided suggestions to improve grammar, enhance clarity, and ensure an academic tone. The LLM was also used to generate a list of potential titles for inspiration, though the final title was conceived and refined by the authors and not taken directly from any single output. In the implementation phase, an LLM served as a coding assistant by offering code completions and debugging support. However, all final code, experimental design, and validation were implemented and verified exclusively by the authors. It is important to emphasize that LLMs were **NOT** used for core scientific contributions, such as generating research ideas, designing experiments, or conducting the literature review. All conceptual work and experimental design originated solely with the authors.

B. Training Details

This section provides the detailed hyperparameters used for training our three agents. All agents were trained on NVIDIA H800 GPUs.

B.1. Grounding Agent

The Grounding Agent was trained using Reinforcement Learning based on Qwen2.5-Coder-7B-Instruct. Its primary role is to identify the correct database schema entities relevant to the user’s question. The training was conducted using the Verl framework (Sheng et al., 2024). The hyperparameters for the RL training and data generation phases are detailed in Table 4.

Training Steps and Convergence: We trained the Grounding Agent for **600 steps** with a batch size of 64. During training, we observed clear stability and convergence patterns in the reward curves; the reward consistently increased and then plateaued, indicating that the policy was successfully optimized.

B.2. Generation Agent

The Generation Agent was also trained using Reinforcement Learning based on Qwen2.5-Coder-7B-Instruct, leveraging a training framework adapted from SkyRL (Cao et al., 2025). This agent is responsible for generating the reasoning SQL query trajectories. Its training and data generation hyperparameters are identical to those of the Grounding Agent, as shown in Table 4.

Training Steps and Convergence: This agent was trained for **160 steps** with a batch size of 64. Similar to the Grounding Agent, the reward curve demonstrated stable convergence within this efficient training phase.

Table 4. Hyperparameters for Grounding and Generation Agent RL Training.

Parameter	Value
<i>Training Parameters</i>	
Learning Rate	1×10^{-6}
Batch Size	128
<i>Trajectory Rollout Parameters</i>	
Temperature	0.6
Top-p	0.95

Table 5. Hyperparameters for Validation Agent Dataset Generation.

Parameter	Value
Candidates per Question	16
Temperature	0.7
Top-p	0.9
Top-k	50

B.3. Validation Agent

The Validation Agent was trained via Supervised Fine-tuning (SFT) to select the best SQL query from the candidates generated by the Generation Agent. We performed a full-parameter fine-tuning of the Qwen2.5-Coder-7B-Instruct model (Hui et al., 2024) using the Llama Factory framework. The SFT training hyperparameters are listed in Table 6, and the parameters for generating its training dataset are in Table 5.

Table 6. Hyperparameters for Validation Agent SFT.

Parameter	Value
Base Model	Qwen2.5-Coder-7B-Instruct
Epochs	3
Learning Rate Scheduler	Cosine
Initial Learning Rate	1×10^{-5}
Effective Batch Size	4
Per-device Batch Size	1
Gradient Accumulation	2 steps
Precision	bf16
Optimization	DeepSpeed ZeRO Stage 3

C. Dataset

C.1. Training Dataset

Our training data is derived from the Bird benchmark, which comprises 9,428 question-SQL pairs. To ensure high quality, we first filtered this dataset by removing samples flagged as incorrect (Pourreza et al., 2025b; Li et al., 2024b) by both Gemini-2.5-pro and GPT-4o, resulting in a clean set of 8,036 training examples. From this set, we constructed the fine-tuning data for the grounding task. For each of the 8,036 question-database pairs, we generated a distinct training instance for every table within that database. This process resulted in a large-scale dataset of 90,102 individual data points. For each point, the ground truth—whether a table is relevant and which of its columns are used—was programmatically extracted from the gold SQL query using the SQLGlot parser.

We constructed a specialized dataset for training the Verifier via Supervised Fine-Tuning (SFT). First, for each question in our filtered BIRD training set, we used both our fine-tuned Generator agent and the initial base model to perform inference, generating a diverse pool of 16 candidate trajectories per question. This ensures the Validation Agent is exposed to a wide range of reasoning paths, both correct and flawed. From this pool, we curated a preference dataset by selecting one positive example (a trajectory leading to a correct execution result) and one negative example (a trajectory leading to an incorrect result) for each question. We mix the order of correct and incorrect trajectories in each pair at random to prevent order bias during training. Since the number of cases containing both correct and incorrect trajectories is limited, some questions yield only flawed trajectories. In such cases, we add the ground truth SQL query in the prompt as a suggestion to help the model generate proper trajectories. We applied best-of-N and worst-of-N (Gui et al., 2024) strategies to select both positive and negative examples. This process yielded a final dataset of approximately 16,000 training instances. Each instance is a triplet containing the user’s question, the full interaction trajectory (including all [Think], [SQL], and [Observation] steps), and the final execution result.

C.2. Evaluation Dataset

BIRD is a large-scale, realistic benchmark designed to evaluate modern Text-to-SQL systems. It features complex databases (33.4 GB across 95 databases), questions from 37 professional domains, and imperfect real-world data values requiring robust handling. BIRD uniquely emphasizes the generation of both correct and efficient SQL queries, making it an ideal testbed for our framework. Our primary evaluations are performed on its development set, which contains 1,534 examples.

Spider 1.0 is a comprehensive, cross-domain benchmark containing 10,181 questions and 5,693 unique complex SQL queries across 200 multi-table databases. It serves as a standard for evaluating cross-domain Text-to-SQL performance. For our evaluation, we use the official test set, which includes 2,147 examples.

Spider-DK, an extension of Spider, is designed specifically to test a model’s ability to handle queries requiring implicit domain knowledge. It comprises samples from the Spider development set that were manually modified to depend on real-world information for correct interpretation. This benchmark simulates scenarios where user queries rely on specific domain context. We evaluate our model on the Spider-DK test set, which contains 535 examples.

SParC (Yu et al., 2019) is a conversational and cross-domain Text-to-SQL benchmark in which each user question depends on the preceding interaction context. We use it as an additional robustness check because it stresses both cross-domain

transfer and multi-turn understanding.

D. Additional Out-of-Domain Robustness

To complement the main-paper results on Spider-test and Spider-DK, we further evaluate MARS-SQL on SParC. Table 7 shows that our model remains strong across these out-of-domain settings, including a particularly large gain on SParC, a benchmark that stresses contextual and interaction-heavy reasoning.

Table 7. Additional out-of-domain robustness results. All numbers are execution accuracy (%).

Model	Spider-test	Spider-DK	SParC
GPT-O3-mini	78.82	71.77	67.0
GPT-5	77.10	66.92	61.8
Qwen2.5-7B	75.87	61.31	64.1
MARS-SQL (Ours)	89.75	78.13	85.78

E. Training Efficiency and Resource Analysis

To address concerns regarding the computational resources required for our multi-agent framework, we provide a detailed breakdown of the training time and a comparative analysis of data efficiency. All experiments were conducted on a node equipped with $4 \times$ NVIDIA H800 GPUs.

E.1. Computational Cost Breakdown

Contrary to the perception that training multiple agents is prohibitively resource-intensive, our framework is designed for rapid convergence. As detailed in Table 8, the entire specialized training pipeline—including the SFT for the Validation Agent and GRPO-based Reinforcement Learning for both the Grounding and Generation Agents—completes in approximately **13 hours**. This represents a modest one-time computational cost, especially considering the significant performance gains achieved.

Table 8. One-Time Training Cost breakdown on $4 \times$ NVIDIA H800 GPUs.

Agent	Method	Training Steps	Batch Size	Est. Training Time
Validation Agent	SFT	~10k	4	1 h
Grounding Agent	GRPO	600	64	4 h
Generation Agent	GRPO	160	64	8 h
Total				~13 h

E.2. Data Efficiency and Comparative Analysis

The efficiency of MARS-SQL stems from its ability to learn diverse reasoning and self-correction behaviors through interaction and self-play, rather than relying on massive-scale supervised datasets.

Table 9 compares our framework against standard single-agent SFT approaches. While standard SFT on the BIRD training set (12k examples) takes only 2 hours, it yields a significantly lower execution accuracy (EX) of 61.08%. Scaling up SFT, as seen in methods like OminiSQL (utilizing 2.5M examples), requires approximately 20 days of training yet only reaches 64.50% EX.

In contrast, MARS-SQL achieves a state-of-the-art EX of **77.84%** using only **35k** LLM-labeled examples and 13 hours of training. To match this performance level using a single-agent SFT-only paradigm, we conservatively estimate—based on scaling laws—that it would require approximately **15 million** synthetic examples and **3–4 months** of training time on the same hardware. Thus, our multi-agent RL framework offers orders of magnitude better data and compute efficiency.

Table 9. Cost and efficiency analysis compared with single-agent SFT baselines on Bird-Dev.

Method	Annotation Source	Data Size	Training Time (wall)	Dev EX (%)
Original (Baseline)	—	—	—	54.56
SFT on BIRD-train	Human	12,000	~2 h	61.08
Large SFT (e.g., OminiSQL)	LLM + Human	2,500,000	~20 days	64.50
MARS-SQL (Ours)	LLM	35,000	~13 h	77.84

F. Inference Efficiency and Practicality Analysis

In this section, we provide a comprehensive analysis of the efficiency and practicality of the MARS-SQL framework. Our goal is not to argue that all systems should be compared as if they had identical product characteristics. Instead, we characterize the end-to-end cost of MARS-SQL and analyze whether its additional test-time compute translates into meaningful accuracy gains compared with both Text-to-SQL baselines and general reasoning models. Since **MARS-SQL** is deployed as three specialized 7B agents, we additionally report the total inference-time parameter footprint (~21B) and the practical VRAM implications of sequential versus fully online deployment. Our analysis covers five aspects: (1) baseline latency and token cost, (2) memory footprint, (3) performance under normalized time budgets, (4) adjustable cost–accuracy trade-offs, and (5) potential system-level optimizations.

F.1. Baseline Time and Token Cost Analysis

We first present the latency breakdown for our standard SOTA-performing configuration ($N_g = 8$ trajectories, $N_v = 8$ validation samples). As shown in Table 10, the average end-to-end latency is 22.12 seconds per query to achieve 77.84% accuracy.

Table 10. Average End-to-End Latency Analysis of MARS-SQL on the BIRD dev set (Hardware: 1x A6000, num_cpus=32). Times represent the average latency to generate one SQL query.

Stage	Avg. Time (s)	Description
1. Grounding Agent	0.78s	Aggregated calls (1 per table)
2. Generation Agent	18.77s	Generating $N_g = 8$ trajectories
3. Validation Agent	2.58s	Validating $N_g = 8$ trajectories ($N_v = 8$ samples each)
<i>Ref: SQL Exec Time</i>	<i>(2.37s)</i>	<i>Avg. time to execute the ground truth SQL</i>
Total (End-to-End)	22.12s	Sum of all stages

The token consumption is analyzed in Table 11. The Generation Agent, utilizing a multi-turn “Think-Act-Observe” loop, accounts for the majority of the token usage.

Table 11. Average Token Cost Analysis per Query.

Stage	Avg. Tokens	Description
1. Grounding Agent	875	Prompt + Schema + Question + Output
2. Generation Agent	9,200	$N_g = 8 \times (\text{Prompt} + \text{Schema} + \text{Question} + \text{Traj.})$
3. Validation Agent	3,250	$N_g = 8 \times N_v = 8 \times (\text{Prompt} + \text{Trajectory})$
Total (Avg.)	13,325	Sum of all components

F.2. Memory Footprint and Deployment Implications

The latency and token analyses above do not fully characterize deployment cost, because **MARS-SQL** is a three-agent system. We therefore report the memory footprint of the full pipeline under a HuggingFace-based sequential deployment that avoids additional engine-level pre-allocation. In this setup, the three agents are executed sequentially (Grounder → Generator → Validation), so the peak VRAM is close to that of a single active 7B agent rather than the sum of all three models loaded simultaneously.

This distinction matters for interpreting the “7B-class” description. Each module is indeed built from a 7B model, but

Table 12. VRAM footprint of MARS-SQL under different deployment assumptions.

Deployment Setting	VRAM	Note
Single 7B checkpoint (BF16 weights on disk)	15.22 GB	Static checkpoint size
Sequential inference peak (2K context)	≈18.5 GB	One active agent at a time
Estimated full online deployment (3 agents resident)	≈46 GB	Sum of weights + runtime buffers
A6000 + SGLang (observed)	≈41 GB	Includes engine pre-allocation
A6000 + vLLM (observed)	≈47 GB	Includes engine pre-allocation

the total inference-time system footprint is approximately 21B parameters. At the same time, peak VRAM in our default sequential execution is much closer to a single 7B deployment than to loading all three modules simultaneously. In other words, parameter count and peak memory are related but not identical metrics in our setting.

We also emphasize that the higher VRAM observed with SGLang or vLLM on A6000 GPUs reflects system-level optimization choices such as memory reservation and kernel/workspace pre-allocation for throughput, rather than the intrinsic minimum memory required by the model weights themselves.

F.3. Accuracy–Latency Trade-offs

We next analyze whether the additional latency of MARS-SQL translates into meaningful performance gains. Our GPT-5.2 comparison is *not* intended as a direct system-level comparison with MARS-SQL. Instead, it is used to illustrate the efficiency of test-time compute trade-offs within a similar model family: additional “thinking” time is valuable when it yields sufficiently large accuracy gains. Table 13 summarizes this comparison under a unified local profiling setup, while Figure 5 visualizes the same trend as an accuracy–latency frontier. From this perspective, MARS-SQL follows the same principle but delivers a stronger return on added latency for Text-to-SQL. To further address the concern that a strong single-agent baseline might catch up given a comparable inference budget, we also evaluate a high-budget SQL-R1 (Ma et al., 2026) setting with 64 sampled outputs and majority voting.

Table 13. Accuracy–latency trade-off under a unified local profiling setup. “Total Time” includes generation and SQL execution.

Method	Total Time (s)	EX Acc. (%)
GPT-5.2-instant	6.0	65.8
GPT-5.2-thinking	15.0	68.6
SQL-R1	6.5	66.43
SQL-R1 + 64 Samples + Voting	33.8	67.34
MARS-SQL (Ours)	22.1	77.8

This interpretation is further supported by the incremental accuracy-per-second return of extra test-time compute. GPT-5.2-instant → GPT-5.2-thinking yields approximately +0.31 acc/sec, while SQL-R1 → MARS-SQL yields approximately +0.73 acc/sec. Importantly, even when SQL-R1 is given a substantially larger test-time budget than MARS-SQL (33.8s vs. 22.1s), its final majority-voted accuracy remains only 67.34%. In other words, MARS-SQL does not simply spend more time; it converts additional latency into larger execution-accuracy gains on challenging Text-to-SQL reasoning, which is also visible in Figure 5.

To further separate this observation from any product-level benefits unique to proprietary systems, Table 14 reports the same style of comparison on open-weight reasoning models. Across Qwen3-30B-A3B, gpt-oss-20B, and GPT-5.2, enabling more deliberative inference consistently trades latency for higher accuracy. MARS-SQL occupies the high-accuracy end of the same trend, and also shows the highest accuracy gain per added second among the comparisons reported here.

F.4. Performance Comparison Under Normalized Time Budget

The accuracy–latency curves above compare several operating regimes, but one reviewer-facing concern is even more direct: can a strong single-agent system match MARS-SQL if it is granted the same wall-clock budget? To answer this, we additionally compare against self-consistency baselines under an equal time budget of approximately 22 seconds, matching the standard MARS-SQL configuration ($N_g = 8$, $N_v = 8$). The SFT baseline uses the budget to generate 16 independent samples, while the closed-source baseline uses the budget to make four independent API calls; both select the final answer

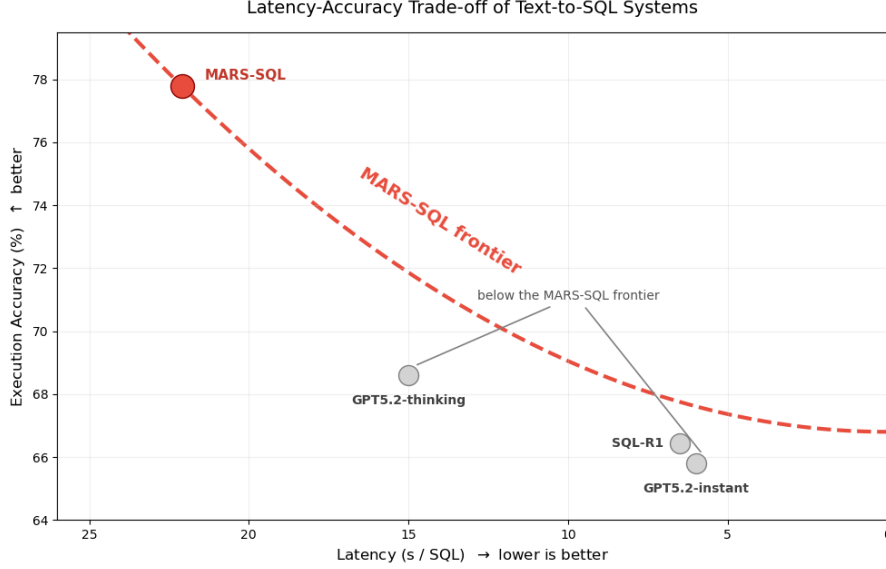


Figure 5. Accuracy–latency Pareto plot using the methods shown in Table 13. MARS-SQL lies on the high-accuracy frontier and extends the same trend observed in reasoning models: more test-time compute can be worthwhile when it buys substantially better execution accuracy.

Table 14. Open-weight and reasoning-model comparisons under the same profiling setup.

Method	Time (s)	Acc. (%)	Note
Qwen3-30B-A3B (No Thinking)	1.3	48.11	—
Qwen3-30B-A3B (With Thinking)	15.8	54.69	+6.58% ACC for +14.5s
gpt-oss-20B (No Thinking)	3.6	52.74	—
gpt-oss-20B (With Thinking)	12.4	55.80	+3.06% ACC for +8.8s
GPT-5.2-instant (No Thinking)	6.0	65.8	—
GPT-5.2-thinking (With Thinking)	15.0	68.6	+2.8% ACC for +9.0s
SQL-R1	6.5	66.43	—
SQL-R1 + 64 Samples + Voting	33.8	67.34	+0.91% ACC for +27.3s
MARS-SQL (Ours)	22.1	77.8	+11.4% ACC for +15.6s vs. SQL-R1

by self-consistency.

Table 15. Accuracy comparison under a normalized time budget (≈ 22 s).

Method	Configuration	Avg. Latency	EX Acc. (%)
Qwen-SFT (Self-Consistency)	SFT + 16 samples	≈ 22.0 s	64.2
GPT-5 (Self-Consistency)	4 API calls	≈ 22.0 s	69.3
MARS-SQL (Ours)	Multi-agent RL	22.12s	77.84

As shown in Table 15, these baselines fail to match the performance of MARS-SQL even when granted an equivalent time budget. This supports the view that the gain comes from the structured multi-agent reasoning workflow and trained trajectory validation, rather than merely from spending more inference time.

F.5. Adjustable Cost–Accuracy Trade-offs

The latency reported in Table 10 represents a performance-oriented configuration. MARS-SQL allows for flexible deployment by adjusting the number of generation trajectories (N_g) and validation samples (N_v). Table 16 illustrates these trade-offs.

Users can select a “Fast” setting to achieve a ≈ 3 -second response that still surpasses the greedy SFT baseline, or invest more computational resources for maximum performance.

Table 16. Tunable Cost-Accuracy Curve for MARS-SQL.

Mode	Params (N_g, N_v)	Latency	Acc. (%)	Characteristic
Fast	(1, 1)	3.1s	68.71%	High speed, outperforms SFT
Balanced	(4, 4)	11.5s	74.90%	Balanced cost-benefit
SOTA (Ours)	(8, 8)	22.1s	77.84%	Maximum accuracy
Over-Sampling	(16, 8)	42.8s	77.84%	Diminishing returns

We do not claim that MARS-SQL is universally cheaper than every single larger model. Rather, our claim is that its accuracy gains are not explained by parameter scaling alone, and that its deployment trade-offs can be made explicit through latency, token, and VRAM reporting.

F.6. System-Level Optimization

The latency metrics presented above assume a sequential, single-query execution, serving as a conservative upper bound. In practical multi-user deployments, **MARS-SQL** can achieve higher throughput through system-level optimizations:

1. **Pipeline Parallelism:** The Grounder, Generator, and Validator agents can process different queries in parallel, creating a pipeline for incoming requests.
2. **Batched Validation:** The $N_g \times N_v$ validation calls are embarrassingly parallel and can be fused into batched requests to reduce amortized costs.

Table 17. Sequential vs. System-Optimized Deployment (Conceptual Comparison).

Deployment	Execution Pattern	Est. Latency	Est. Throughput
Sequential (No optimization)	Grounder $\rightarrow$ Generator $\rightarrow$ Validator (End-to-End)	$\approx 22.1s$	≈ 2.7 queries/min
System-Optimized (Pipeline + Batch)	Pipelined stages; batched validation	$\approx 12\text{--}15s$	$\approx 4\text{--}5$ queries/min

Table 17 estimates that with these optimizations, the effective per-query latency can be reduced by approximately 40–60%, significantly improving throughput on a single GPU node.

G. Architecture versus Parameter Scaling

A critical consideration when evaluating multi-agent frameworks is whether the observed performance improvements are intrinsically tied to the novel architectural design, or merely a byproduct of increased parameter capacity at inference time. To decouple our methodological contributions from pure scaling effects, we conduct three complementary analyses. First, we benchmark **MARS-SQL** against a parameter-matched ($\sim 21B$) ensemble constructed from highly competitive 7B Text-to-SQL models (Table 18). Second, we investigate the scaling behavior of the core framework by expanding the Generation Agent from 7B to 14B parameters, maintaining the identical Reinforcement Learning (RL) training recipe (Table 19). Third, to directly address the possibility that a single larger model could absorb all three roles through increased capacity, we train single-model multi-task baselines that use one shared model to perform grounding, generation, and validation (Table 20).

Table 20. Single-model multi-task baselines on BIRD-dev. A single shared model is SFT-trained to perform all three roles: grounding, generation, and validation.

System Configuration	Model Size	EX Acc. (%)
Single-model multi-task (SFT)	14B	55.67
Single-model multi-task (SFT)	32B	57.82
MARS-SQL (Ours)	3 $\times$ 7B	77.84

Table 18. Comparison against a strong ~ 21 B ensemble on BIRD-dev. The ensemble combines Arctic-text2sql-7B, SQL-R1, and OmniSQL-7B, generates eight queries per model, and selects the final answer by majority voting.

System Configuration	Total Parameters	EX Acc. (%)
Arctic-text2sql-7B (Single)	7B	67.73
SQL-R1 (Single)	7B	66.03
OmniSQL-7B (Single)	7B	65.31
SOTA 7B Ensemble (Voting)	~ 21B	72.10
MARS-SQL (Ours)	~ 21B	77.84

Table 19. Scaling the generator versus using multi-agent specialization on BIRD-dev.

System Configuration	Core Model	EX Acc. (%)	Gain over 7B Base
Generator Only (Base)	7B	66.37	–
Generator Only (Scaled)	14B	67.92	+1.55
MARS-SQL (Ours)	3×7B	77.84	+11.47
MARS-SQL (Scaled Gen)	$14\text{B} + 2 \times 7\text{B}$	78.62	+12.25

These results support three conclusions. First, as shown in Table 18, simply stacking three strong 7B Text-to-SQL models into a ~ 21 B ensemble reaches 72.10%, which is still 5.74 points below **MARS-SQL**. Second, Table 19 shows that scaling the generator from 7B to 14B improves the single-agent baseline by only 1.55 points, whereas introducing the specialized multi-agent workflow improves it by 11.47 points. Third, Table 20 demonstrates that a single shared model trained to absorb all three roles performs substantially worse even at larger scales: the 14B and 32B multi-task baselines reach only 55.67% and 57.82% EX, respectively. Together, these findings indicate that the dominant source of improvement is the role-specialized interactive workflow, not raw parameter count alone.

H. Multi-turn Generation

Evolution of Interaction Turns: To understand the impact of RL training on the agent’s reasoning efficiency, we analyzed the evolution of rollout lengths during the training process. In our setting, each “Think–Act–Observe” cycle corresponds to one database interaction turn, making the average number of interaction turns a proxy for rollout length.

Grounding Agent (Single-turn): The rollout length (token count) exhibited a mild U-shaped pattern. Initially, the output became more concise, followed by a slight lengthening to include only essential schema information. This reflects a refinement of the policy towards precise schema selection rather than reasoning from scratch.

Generation Agent (Multi-turn): A distinct trend was observed where the average number of interaction turns consistently decreased and stabilized at a lower level. This indicates that the agent learned to solve problems more directly and recognized when to terminate the search efficiently. This efficiency gain is quantitatively supported by the evaluation on the BIRD-dev set (with a maximum of 5 turns), as shown in Table 21. The RL-trained agent significantly reduces the average turns across all difficulty levels compared to the base model.

Table 21. Comparison of Average Interaction Turns on BIRD-dev (Max Turns = 5) before and after RL training.

Model	Avg. Turns (Challenging)	Avg. Turns (Moderate)	Avg. Turns (Simple)
Before RL (Base Model)	2.90	2.67	2.27
After RL (Generation Agent)	1.82	1.71	1.45

Furthermore, analyzing the specific distribution of turns reveals that the agent learns an adaptive and non-wasteful strategy. As presented in Table 22, while the agent retains the capacity to use multiple turns for complex reasoning, it solves the vast majority of problems (1,116 cases) in a single interaction. Crucially, for the “long-tail” of more difficult queries, the agent robustly applies deeper reasoning, utilizing up to 5 or more turns to arrive at the correct solution. This distribution confirms that the agent is not bound by arbitrary limits but instead dynamically decides the necessary reasoning depth for each specific query.

Figure 6 provides a concrete example of the agent’s interactive and self-correcting workflow. The agent initially generates a

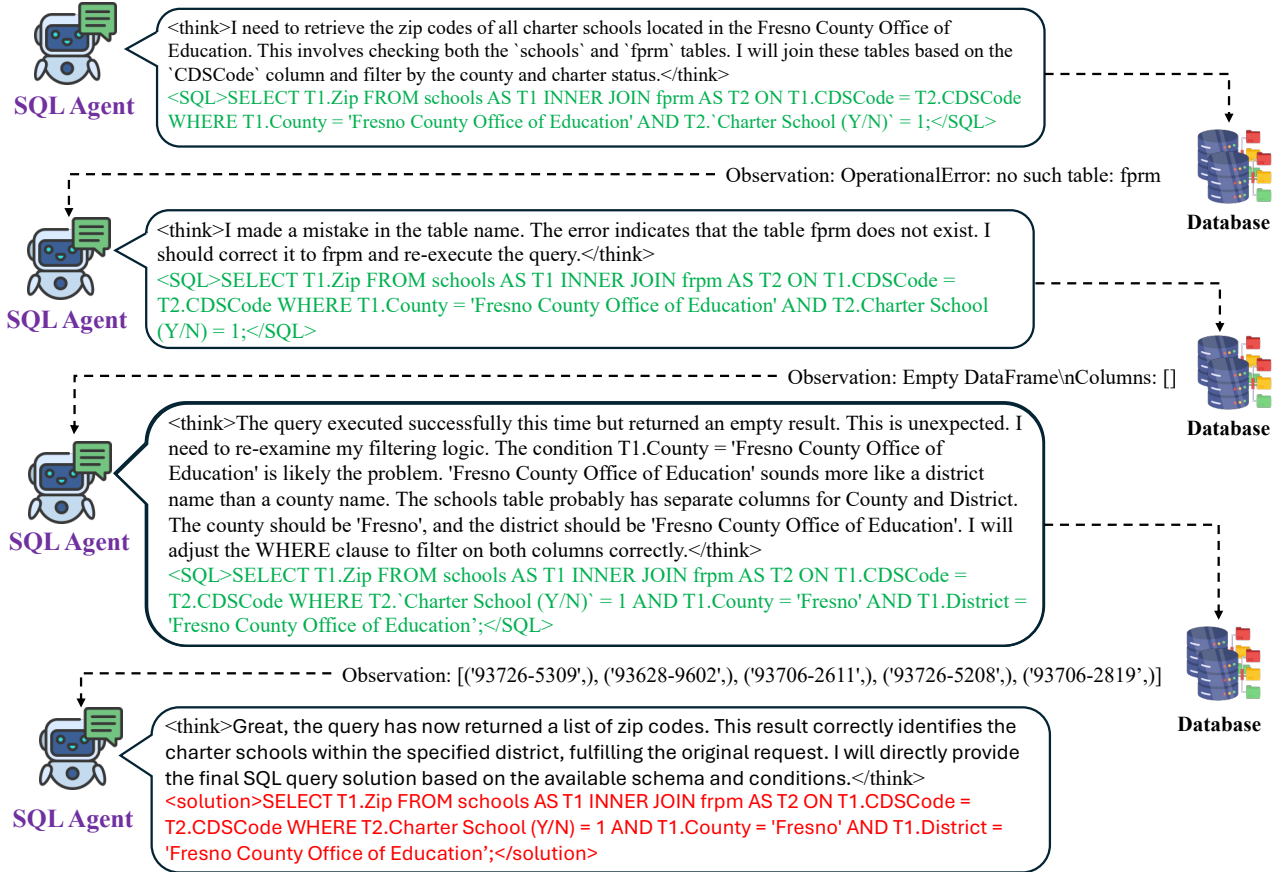
Table 22. Distribution of Interaction Turns Used by the Generation Agent on BIRD-dev.

# of Interaction Turns	# of Examples
1	1,116
2	174
3	105
4	88
5+	51

query with a typographical error in a table name ‘fprm’. Upon receiving an ‘OperationalError’ from the database, it correctly identifies the mistake in its next thought process, corrects the table name to ‘frpm’, and re-executes the query. The second attempt returns an empty result, prompting the agent to re-evaluate its filtering logic. It deduces that it incorrectly used a district name as a county name and refines the ‘WHERE’ clause accordingly. This final attempt successfully retrieves the correct data, leading the agent to output the validated query as its final solution.

Figure 6. Example of interactive process in the generation.

Q: Please list the zip code of all the charter schools in Fresno County Office of Education.



I. Qualitative Analysis: Reasoning Beyond Valid Execution

A core feature of our Generation Agent is its learned policy to evaluate the *semantic content* and *plausibility* of execution results, rather than merely checking for syntax errors. This allows the agent to continue reasoning even when the database

returns a non-empty, non-error result. Below, we present two real-world case studies from the BIRD-dev set illustrating this capability.

I.1. Scenario 1: Complex Problem Decomposition

In complex queries, a successful execution often represents only an intermediate step in a larger plan. The agent effectively decomposes the problem, using the observation from the first step to inform the subsequent logic.

Question: Consider the average difference between K-12 enrollment and 15-17 enrollment of schools that are locally funded, list the names and DOC type of schools which has a difference above this average.

Turn 1 (Think): To solve this problem, I need to follow these steps: 1) Calculate the average difference between K-12 enrollment and 15-17 enrollment for all schools. 2) Filter the schools based on the funding type. 3) Calculate the difference for filtered schools. 4) Identify schools above the average.

Turn 1 (SQL):

```
SELECT AVG("Enrollment (K-12)" - "Enrollment (Ages 5-17)") FROM frpm;
```

Turn 1 (Observation): 16.261833

Turn 2 (Think): (*The agent recognizes this is an intermediate value, not the final answer.*) The average difference is approximately 16.26. Now, I will filter the schools based on the funding type (locally funded) and calculate the difference for these schools against this average.

Turn 2 (SQL):

```
SELECT T2.School, T2.DOC FROM frpm AS T1 JOIN schools AS T2 ON T1.CDSCode = T2.CDSCode
↪ WHERE T2.FundingType = 'Locally funded' AND (T1."Enrollment (K-12)" - T1."
↪ Enrollment (Ages 5-17)") > 16.26;
```

Turn 2 (Observation): [(‘Circle of Independent Learning’, ‘54’), (‘Learning Community Charter’, ‘00’)]

Turn 3 (Think): The query executed successfully and returned non-empty results. I will now combine the logic into a single nested query for the final answer.

Turn 3 (Final Solution):

```
SELECT T2.School, T2.DOC FROM frpm AS T1 INNER JOIN schools AS T2 ON T1.CDSCode = T2.
↪ CDSCode WHERE T2.FundingType = 'Locally funded' AND (T1."Enrollment (K-12)" - T1."
↪ Enrollment (Ages 5-17)") > (SELECT AVG(T3."Enrollment (K-12)" - T3."Enrollment (
↪ Ages 5-17)") FROM frpm AS T3 INNER JOIN schools AS T4 ON T3.CDSCode = T4.CDSCode
↪ WHERE T4.FundingType = 'Locally funded')
```

I.2. Scenario 2: Data Plausibility and Sanity Checking

The agent also learns to perform “sanity checks.” If a syntactically correct query returns implausible data (e.g., placeholder values), the agent triggers a self-correction loop to verify and refine the query before finalizing the answer.

Question: Which cities have the top 5 lowest enrollment number for students in grades 1 through 12?

Turn 1 (Think): I need to find the cities with the top 5 lowest enrollment numbers. I will join the tables and order by enrollment ascending.

Turn 1 (SQL):

```
SELECT T2.City, T1."Enrollment (K-12)" FROM frpm AS T1 INNER JOIN schools AS T2 ON T1.
↪ CDSCode = T2.CDSCode ORDER BY T1."Enrollment (K-12)" ASC LIMIT 5;
```

Turn 1 (Observation): [(‘Markleeville’, 1.0), (‘Oakland’, 1.0), (‘Palermo’, 1.0)]

Turn 2 (Think): (*The agent detects a potential data anomaly.*) The result shows enrollment numbers are very low (1.0), which might indicate an error in the data or the need to filter out non-enrollment entries. Let's check the data to ensure accuracy.

Turn 2 (SQL):

```
SELECT CDSCode FROM frpm WHERE "Enrollment (K-12)" != 1.0 LIMIT 10;
```

Turn 2 (Observation): [('01100170130419'), ('01100170112607') ...]

Turn 3 (Think): The check confirms that there are legitimate records with other values, and '1.0' likely represents missing or placeholder data. I need to filter these out to get accurate city names.

Turn 3 (SQL):

```
SELECT T2.City FROM frpm AS T1 INNER JOIN schools AS T2 ON T1.CDSCode = T2.CDSCode WHERE
↪ T1."Enrollment (K-12)" != 1.0 ORDER BY T1."Enrollment (K-12)" ASC LIMIT 5;
```

Turn 3 (Observation): [('Bishop'), ('Kingsburg'), ('Oakland'), ('Williams'), ('Compton')]

Turn 4 (Think): Now, the cities with the lowest enrollment numbers are identified and the data looks plausible. I can provide the final SQL query.

Turn 4 (Final Solution):

```
SELECT T2.City FROM frpm AS T1 INNER JOIN schools AS T2 ON T1.CDSCode = T2.CDSCode WHERE
↪ T1."Enrollment (K-12)" != 1.0 ORDER BY T1."Enrollment (K-12)" ASC LIMIT 5;
```

I.3. Scenario 3: Overcoming Sub-optimal Logic and Boundary Errors

This case demonstrates the agent's ability to recover from hidden logical and semantic errors, specifically regarding boundary conditions and offset calculations. Even when a query executes perfectly without syntax errors, the agent evaluates the mathematical alignment between its SQL logic and the user's intent, iteratively refining its approach to overcome sub-optimal reasoning.

Question: What is the eligible free rate of the 10th and 11th schools with the highest enrolment for students in grades 1 through 12?

Turn 1 (Think): The question asks for the eligible free rate for the 10th and 11th schools based on enrollment. I need to calculate this rate, sort the schools by enrollment in descending order, and select the 10th and 11th positions. I will use the LIMIT clause to extract these two specific rows.

Turn 1 (SQL):

```
SELECT "Free Meal Count (K-12)" / "Enrollment (K-12)" as rate FROM frpm ORDER BY "
↪ Enrollment (K-12)" DESC LIMIT 1, 2;
```

Turn 1 (Observation): [(0.815746,), (0.338349,)]

Turn 2 (Think): (*The agent detects a sub-optimal logic in pagination.*) The query successfully returned results, but the logic LIMIT 1, 2 is incorrect for finding the 10th and 11th schools. In SQL, the offset starts at 0, so an offset of 1 skips only the 1st row and returns the 2nd and 3rd rows. Furthermore, to prevent potential integer division truncation, I should cast the numerator to a real number. I need to adjust the offset to 9 to correctly target the 10th and 11th schools.

Turn 2 (SQL):

```
SELECT CAST("Free Meal Count (K-12)" AS REAL) / "Enrollment (K-12)" as rate FROM frpm
↪ ORDER BY "Enrollment (K-12)" DESC LIMIT 9, 2;
```

Turn 2 (Observation): [(0.134436,), (0.290589,)]

Turn 3 (Think): The query now orders the rates by enrollment in descending order and uses the correct pagination boundary. `LIMIT 9, 2` correctly skips the top 9 schools and selects the 10th and 11th rows. The data type casting also ensures robust calculation. The results are now logical and correct.

Turn 3 (Final Solution):

```
SELECT CAST("Free Meal Count (K-12)" AS REAL) / "Enrollment (K-12)" FROM frpm ORDER BY "
↪ Enrollment (K-12)" DESC LIMIT 9, 2;
```

I.4. Scenario 4: Recovery from Schema Hallucination and Ambiguity

This case illustrates the agent’s capacity to recover from schema hallucinations (e.g., mistyping or misremembering table names) and simultaneously resolve semantic ambiguities. When confronted with a database execution error, the agent not only corrects the structural typo but also proactively refines the logical filtering conditions based on a deeper understanding of the schema constraints.

Question: Please list the zip code of all the charter schools in Fresno County Office of Education.

Turn 1 (Think): I need to retrieve the zip codes of all charter schools within the Fresno County Office of Education. I will join the `schools` table with the `frpm` table on the `CDSCode` and filter by the county and charter school status.

Turn 1 (SQL):

```
SELECT T1.Zip FROM schools AS T1 INNER JOIN frpm AS T2 ON T1.CDSCode = T2.CDSCode WHERE
↪ T1.County = 'Fresno County Office of Education' AND T2."Charter School (Y/N)" = 1;
```

Turn 1 (Observation): Operational Error: no such table: frpm

Turn 2 (Think): (*The agent processes the database error and reviews the schema.*) I made a typo in the table name; it should be `frpm` instead of `fprm`. Furthermore, 'Fresno County Office of Education' likely refers to the school `District` rather than the `County` itself, as the county name is typically just 'Fresno'. I will correct the table hallucination and refine the `WHERE` clause to check both `County` and `District` for precise filtering.

Turn 2 (SQL):

```
SELECT T1.Zip FROM schools AS T1 INNER JOIN frpm AS T2 ON T1.CDSCode = T2.CDSCode WHERE
↪ T2."Charter School (Y/N)" = 1 AND T1.County = 'Fresno' AND T1.District = 'Fresno
↪ County Office of Education';
```

Turn 2 (Observation): [('93726-5309',), ('93628-9602',), ('93706-2611',), ('93726-5208',), ('93706-2819',)]

Turn 3 (Think): The query executed successfully and returned a valid list of zip codes. The table name was correctly resolved, and the district/county filtering accurately identified the target schools. I can now provide the final solution.

Turn 3 (Final Solution):

```
SELECT T1.Zip FROM schools AS T1 INNER JOIN frpm AS T2 ON T1.CDSCode = T2.CDSCode WHERE
↪ T2."Charter School (Y/N)" = 1 AND T1.County = 'Fresno' AND T1.District = 'Fresno
↪ County Office of Education';
```

J. Dataset-Wide Controlled Isolation

To complement the component ablation in the main paper, we perform a dataset-wide controlled-isolation analysis on the full BIRD-dev set (1,534 queries). The goal is to quantify how many additional queries are resolved by each mechanism when starting from a strict baseline. We use the RL Generator only with $T=1$ and greedy decoding as the baseline configuration, which achieves 62.78% execution accuracy.

Two observations follow from Table 23. First, generative verification is the single largest isolated contributor, which is consistent with the strong gains observed in our selection-method analysis. Second, the RL interactive policy contributes 44

Table 23. Dataset-wide controlled isolation on BIRD-dev. Each row attributes additional solved queries beyond the strict RL-Generator baseline.

Configuration / Mechanism	Extra Solved Cases	Gain
Strict baseline: RL Generator only ($T=1$, Greedy)	—	62.78% EX
+ Schema pruning	56	+3.63%
+ RL interactive reasoning	44	+2.89%
+ Generative verification / reranking	131	+8.54%
Full MARS-SQL	231	77.84% EX

Table 24. Ablation study on the maximum number of interaction turns (T). We compare the original model against models fine-tuned with different turn limits. For each setting, we report **Greedy** accuracy (from a single output), **Best of 8** accuracy, and **Selected** accuracy (after reranking 8 candidates) with grounding agent. All scores are execution accuracy (%) on the BIRD-dev benchmark.

Model (Trained w/ Max Turns)	Inference ($T=1$)			Inference ($T=5$)			Inference ($T=10$)		
	Greedy(%)	Best of 8(%)	Select(%)	Greedy(%)	Best of 8(%)	Select(%)	Greedy(%)	Best of 8(%)	Select(%)
Original Model (Base)	54.43	77.76	69.69	55.41	77.82	70.34	55.76	77.56	70.07
Trained ($T=1$)	66.41	78.6	72.06	66.95	78.76	72.75	67.60	80.63	74.19
Trained ($T=5$)	67.60	82.19	75.29	69.30	83.7	77.84	68.25	82	76.40
Trained ($T=10$)	67.73	83.61	76.86	69.36	83.95	77.12	69.88	83.89	77.57

additional correct queries under greedy decoding alone, without any sampling or reranking. This directly supports our claim that the model has learned to use execution feedback for self-correction at the policy level, rather than benefiting only from test-time selection.

K. Interaction turns impact

Table 24 presents an ablation study on the impact of the maximum number of interaction turns (T) allowed during training and inference. The study compares the performance of the base model against versions fine-tuned with maximum turn limits of $T=1$, $T=5$, and $T=10$. For each training configuration, we report three execution accuracy metrics on the BIRD-dev benchmark: Greedy (a single output), Best of 8 (the highest score among 8 candidates), and Selected (the score after reranking). The results indicate that increasing the number of turns during fine-tuning consistently improves performance. For example, the model trained with $T=10$ achieves the highest Greedy (69.88%) and Best of 8 (83.89%) accuracy when inferring with 10 turns, demonstrating the value of a larger interaction budget for complex reasoning.

L. Impact of Multiple Candidate Generations

To evaluate the impact of generating multiple candidate trajectories, we conduct a “Best-of- N ” analysis, where N is the number of parallel rollouts. As shown in Table 25, increasing the number of candidates provides a substantial performance boost. This demonstrates that the exploratory nature of our Generator agent is effective at covering the solution space, with the upper-bound performance (Pass@ N) increasing consistently with more samples. The final accuracy, after applying our Generative Validation Agent, also benefits from a larger pool of high-quality candidates to select from.

Table 25. Impact of “Best-of- N ” selection on the BIRD-dev benchmark. **Greedy (Best of 1)** is the execution accuracy of the final selected trajectory. **Best of N** represents the upper-bound performance (Pass@ N), indicating the percentage of times at least one correct trajectory was found among N candidates. Inference parameters: temperature=0.8, top_k=50, top_p=0.7, max_iterations=5.

Selection Strategy	Execution Accuracy (%)
Greedy (Best of 1)	69.30
Best of 2	74.04
Best of 4	79.71
Best of 8	83.76
Best of 16	86.31
Best of 32	87.54

Table 26. The prompt used to guide the agent in the table-level schema linking task. It includes the role description, task instructions, output format examples, and the prefix for the agent’s response.

Prompt for Table-level Schema Linking
User: You are doing table level schema linking. Given a table with schema information and the task, you should think step by step and decide whether this table is related to the task. Your thought process should be enclosed in <code><think></think></code> tags, and your final decision in <code><answer></answer></code> tags. For the answer, first state ‘Y’ for relevant or ‘N’ for not relevant. If relevant, also provide a Python list of the column names you believe are most useful. Example of a final answer format: <pre><answer> Y ["player_name", "team_name", "matches_played"] </answer></pre> or <pre><answer> N </answer></pre> Here is the information for the current task: ### Table Information: <code>{table_info}</code> ### User Question: <code>{task}</code> ### External Knowledge (if any): <code>{external}</code> Assistant: Let me solve this step by step. <code><think></code>

M. Table Level Grounding

Table 26 details the prompt for our RL-trained Schema Grounding Agent, which elicit a step-by-step reasoning process during inference. It instructs the agent to analyze a given table’s schema in the context of the user’s question and any external knowledge. The agent is required to first articulate its analysis within ‘`<think>`’ tags, followed by a final, parsable decision in ‘`<answer>`’ tags. This output must specify the table’s relevance (‘Y’/‘N’) and, if applicable, a Python list of useful columns. This structured format ensures a transparent and predictable output format crucial for our framework.

Table 27 presents recall and precision statistics for our schema grounding agent, comparing our RL-based approach against the base model and a version trained with Supervised Fine-Tuning (SFT). The results clearly demonstrate the superiority of our method, which achieves exceptionally high recall and precision across all benchmarks. On the complex in-domain BIRD-dev set, our primary concern is recall. Our agent achieves a recall of 97.78%, with only 48 examples failing to identify all required schema components, which we consider a highly effective result. Simultaneously, it maintains a high precision of 90.74%, indicating that the selections are not only comprehensive but also accurate. This strong performance extends to the out-of-domain Spider-test and Spider-DK benchmarks, underscoring the robustness of our RL-trained grounding agent.

The practical impact of the Grounding Agent depends on the schema regime. On smaller and cleaner schemas such as Spider, the Generator can often operate with comparatively limited interference, so the end-to-end gain from grounding is smaller. On BIRD-dev, however, the search space is much larger, making schema filtering substantially more valuable. More broadly, this requirement to preserve salient evidence while filtering large amounts of irrelevant context is also consistent with observations from long-form and long-context analysis (He et al., 2026). Table 28 summarizes this contrast.

Table 27. Recall and precision statistics after grounding for Bird-dev, Spider-test and Spider-DK. **Recall** measures the percentage of instances where all required columns were identified. **Precision** measures the ratio of required columns to all selected columns, indicating the selection’s accuracy.

Grounding Model	Bird dev		Spider test		Spider DK	
	Recall (%)	Precision (%)	Recall (%)	Precision (%)	Recall (%)	Precision (%)
Qwen 7B (Base)	68.59	53.45	87.48	69.22	84.25	66.54
Qwen 7B + SFT	74.97	67.01	90.39	78.16	88.60	72.71
Qwen 7B + RL (Ours)	97.78	90.74	98.97	93.62	98.13	91.59

Table 28. Dataset characteristics relevant to the Grounding Agent. Larger schemas create more distractors for downstream generation.

Dataset	Avg. Schema Tokens	Avg. Columns/DB	Avg. Tokens/SQL
Spider	~250	26.82	30.9
BIRD-dev	~950	54.56	18.5

This context also helps explain why the Grounder is more important under semantic ambiguity and domain shift. Spider-DK is specifically designed to break clean lexical matches between questions and schemas, and Table 27 shows that our RL Grounder maintains very high recall and precision there as well. In this sense, grounding is best understood not as a uniformly large gain on every benchmark, but as a component whose importance increases with schema scale, ambiguity, and cross-domain difficulty.

N. Generation Agent Prompt

Table 29 details the comprehensive prompt structure used to guide the agent’s multi-turn generation process. The prompt establishes the agent’s persona as a data science expert and provides all necessary context, including the database schema, external knowledge, and the user’s question. It strictly enforces an output format that requires the agent to vocalize its reasoning within <think> blocks before executing a query in a <sql> block. The database returns feedback in an <observation> block, which the agent uses for subsequent reasoning turns, ultimately providing the final answer in a <solution> block. This iterative structure is designed to facilitate a dynamic, step-by-step problem-solving process.

O. Validation Agent

Our Generative Validation Agent is guided by the prompt detailed in Table 30. The prompt instructs the agent to act as an expert SQL data analyst, with the objective of evaluating the logical correctness of a proposed SQL solution for a given problem. Unlike our previous approach, this prompt no longer constrains the agent to reason about a sampled or truncated database. Instead, it assumes the agent evaluates the query’s validity against the full database schema and context. The prompt structure provides the agent with the user’s question, the candidate SQL solution, and a dedicated field for any relevant “External Knowledge” that might be necessary for a correct evaluation. The output format remains strict, requiring the agent to begin its response with a definitive “Yes” or “No” before any subsequent reasoning.

P. LLM as a judge prompt

The prompt in Table 31 is used for our baseline selection method LLM as a judge. This prompt is designed to guide the model in identifying the optimal SQL query from a set of generated candidates. The agent is explicitly instructed to consider each candidate’s associated reasoning, the SQL query itself, and most crucially, its execution observation on the database. This emphasis on execution results is paramount, as it allows the agent to distinguish between syntactically correct queries and those that truly provide the correct and complete answer to the user’s question, even if a query might appear correct but yields erroneous or empty results. After presenting the user’s question and the formatted candidate solutions (each including reasoning, SQL, and execution output), the prompt concludes with strict instructions for the agent to output only the index number of the single best candidate. In cases of ties, the candidate with the lowest index is to be chosen, ensuring a deterministic selection process.

Prompt Format for SQL Reasoning**Prompt Description:**

You are a data science expert. Below, you are provided with a database schema and a natural language question. Your task is to understand the schema and generate a valid SQL query to answer the question within limited turns. You should breakdown the problem, draft your reasoning process, and generate the solution.

Database Engine:

SQLite

Database Schema:

```
{db_details}
```

This schema describes the database's structure, including tables, columns, primary keys, foreign keys, and any relevant relationships or constraints.

External Knowledge:

```
{external_knowledge}
```

Question:

```
{question}
```

Important Instructions:

- Make sure you only output the information that is asked in the question. If the question asks for a specific column, make sure to only include that column in the SELECT clause, nothing more.
- The generated query should return all of the information asked in the question without any missing or extra information.
- Before generating the final SQL query, please think how to write the query. It should include detailed considerations such as analysing questions, summarizing relevant findings, brainstorming new ideas, verifying the accuracy of the current steps, refining any errors, thinking of how to call SQL tools, and revisiting previous steps.

Output Format (STRICTLY ENFORCED):

- Conduct thinking inside `<think>...</think>` blocks every time you get new observation or information. Start with `<think>...</think>` blocks in your responses as shown in the following example.
- You can use SQL tool written within a single `<SQL>your SQL</SQL>` block to explore or verify. You can't use the format ````SQL ; \n````, you must use the format `<SQL>your SQL</SQL>` to get the output. `<SQL>your SQL</SQL>` block should follow closely behind `<think>...</think>` block. SQL tool output will be shown as dataframe inside `<observation>...</observation>`. Based on this observation, you can think again and refine.
- The returned dataframe will be truncated in 50 rows if observation is too long.
- If you find no further exploration is needed or have only 1 turn left, you MUST directly provide the final SQL query solution inside `<solution>...</solution>`.
- All your responses should be in the `<think>...</think>`, `<sql>...</sql>`, `<observation>...</observation>`, `<solution>...</solution>` blocks.

Example:

Question: how many pigs are in the farm?

Database Schema:

Table: animals

- id (INTEGER, PRIMARY KEY)
- species (TEXT)
- age (INTEGER)
- name (TEXT)

Output:

```
<think>I am querying how many pigs are in the farm. I will begin by checking if
the 'animals' table exists and contains entries with species = 'pig'.</think>
<SQL>SELECT COUNT(*) FROM animals WHERE species = 'pig';</SQL>
<observation>
+-----+
| COUNT(*) |
+-----+
| 12 |
+-----+
</observation>
<think>The result indicates that there are 12 pigs in the farm. Since the
question asks for how many pigs, I can now output the final SQL as the
solution.</think>
<solution>SELECT COUNT(*) FROM animals WHERE species = 'pig';</solution>
```

Table 29. Prompt example for SQL reasoning without memory.

Table 30. The prompt used for the Generative Validation Agent. The agent is framed as a SQL expert and is provided with the problem, the proposed SQL query, and any relevant external knowledge. It evaluates the logical correctness of the query and must provide a final “Yes” or “No” judgment.

Prompt for Generative Validation Agent
User: Task Background: You are an expert SQL data analyst. Your task is to verify if a proposed solution correctly answers a user’s question. Problem: {question} External Knowledge: {external_knowledge} Proposed Solution: {solution_text} Your Task: Based on all the information, is the SQL query in the solution logically correct for answering the question? You must answer with “Yes” or “No” first, before any other text. Is the answer correct (Yes/No)?

Q. Selection method comparison

We compare our proposed Generative Verifier against several strong baselines for trajectory selection, with the results detailed in Table 32. The initial Pass@8 accuracy of our Generator agent’s output establishes the theoretical upper bound for any selection method, as it represents the percentage of questions for which at least one of the eight generated trajectories is correct.

Table 31. The prompt used for the Selection Agent, operating as an LLM judge. It guides the model to select the best SQL query from multiple candidates by evaluating their reasoning, SQL code, and critically, their execution observations. Strict output instructions ensure a direct index selection.

Prompt for Selection Agent (LLM as a Judge)
User: Task Background: You are an expert SQL data analyst. Your task is to select the BEST SQL query that correctly answers a user’s question. You are given several candidates. For each candidate, you will see its reasoning, the SQL query itself, and importantly, the result of executing that query on the database. A query might look correct but return an error or empty/wrong data. You must use the execution observation to make your final decision. Here is the user’s question: {question} Evaluate the following candidates based on ALL available information. Does the “Execution Observation” for a candidate actually answer the user’s question? — {formatted_candidates} — Final Analysis: Considering the reasoning, the SQL code, and especially the execution results, which single candidate provides the most correct and complete answer to the user’s question? Instructions for your response: • Respond with ONLY the index number of the single best candidate. • If multiple candidates produce correct results, select the one with the LOWEST index number. • Do not include any other words, symbols, or explanations. Best candidate index:

Table 32. Ablation study of different selection strategies. The first row, **Pass@8**, shows the baseline execution accuracy (%) of the eight candidate trajectories from our Generator agent before any selection. Subsequent rows report the final accuracy after applying each method to select the best trajectory. **Self-Consistency** picks the most frequent result, **LLM as a Judge** uses GPT-4o/Qwen for selection, and **Ours** uses our fine-tuned 7B Generative Verifier.

Method	Model Size	Bird dev (%)	Spider test (%)	Spider DK (%)
<i>Pass@8 (Generator Output)</i>	-	83.76	90.68	82.06
LLM as a Judge (GPT-4.1)	Unkonwn	75.15	83.47	71.40
LLM as a Judge (Qwen)	7B	70.47	79.60	70.09
Self-Consistency	-	72.93	83.51	73.08
Ours (Generative Verifier)	7B	77.84	89.75	78.13